



MANUAL DE ARQUITECTURA
PLATAFORMA DISTRIBUIDA

JettraStore: Arquitectura Definitiva del Ecosistema de Base de Datos Multimodelo Distribuida en Java 25+

Manual Maestro de Arquitectura, Persistencia Off-Heap,
Consenso Raft, Anillo por Saturación de Memoria y
Operaciones

18 CAPÍTULOS

Versión de Plataforma: Java 25 LTS / Jettra Core 1.0

Tabla de Contenidos

1. Visión General y Filosofía de JettraStore
2. Arquitectura de Almacenamiento y Formato de Archivo `.jettra`
 - 2.1 Persistencia Inmutable LSM y MemTable Off-Heap
 - 2.2 Estructura Binaria de Archivos `.jettra`
 - 2.3 Índices Secundarios Dispersos y Filtros de Bloom
3. Optimizaciones Extremas de la JVM (Java 25+)
 - 3.1 Project Panama: Foreign Function & Memory (FFM) API
 - 3.2 Compact Object Headers (JEP 450)
 - 3.3 Recolección de Basura ZGC de Latencia Sub-Milisegundo
 - 3.4 CRaC y CRIU para Arranque Instantáneo
 - 3.5 Manipulación de Colecciones con `JettraCollection` y Prevención Rigurosa de `OutOfMemoryError`
4. Integración Nativa del Motor Off-Heap `JettraMemory`
 - 4.1 Arquitectura Panama LSM y Segmentos Binarios
 - 4.2 APIs Nativas en `JettraDatabase` y `JettraClient`
 - 4.3 Modos Duales de Almacenamiento: JVM-RAM vs DISK-MEMORY (`JettraMemory`)
5. Mecanismo Dinámico de Anillo Distribuido por Saturación de Memoria
 - 5.1 Detección Preventiva de Umbrales de RAM
 - 5.2 Transición Automática a Motor de Anillo
 - 5.3 Protocolo de Descarga y Rebalanceo Dinámico Off-Heap
6. Topología de Clúster de 3 Nodos y Consenso Raft
 - 6.1 Configuración de Nodos (Líder y Secundarios)
 - 6.2 Canales Raft Sin Bloqueo con Virtual Threads y `jettraGRPC`
7. Seguridad Estricta: Autenticación, Superusuario y `JettraJWT`
 - 7.1 Superusuario Administrativo por Defecto (`admin / admin-jettra`)
 - 7.2 Inviolabilidad y Jerarquía Máxima del Superusuario
 - 7.3 Arquitectura de Tokens `JettraJWT`
8. Soporte Multimodelo y Referencias Cruzadas (Intra e Inter-Engine)
 - 8.1 Los 8 Motores Nativos Integrados
 - 8.2 Referencias Cruzadas Multimodelo
 - 8.3 Estrategias de Carga: Lazy Load vs Eager Load
9. Lenguajes de Consulta: `JettraQueryLanguage` (LQL) y `JettraSQL`
 - 9.1 `JettraQueryLanguage` (LQL) - Estilo Fluent Streams
 - 9.2 `JettraSQL` - Dialecto SQL de Alto Rendimiento
10. Componente de Supervisión Preventiva: `JettraPolice`
 - 10.1 Ciclo de Vida del Daemon Autónomo
 - 10.2 Reglas Preventivas y Acciones Mitigadoras
 - 10.3 Activación y Configuración (`jettrapolice.active`)
 - 10.4 Supervisión Predictiva de Heap y Prevención Autónoma Anti-OOM

- 10.5 Protocolo de Streaming por Chunks y Metadatos de Notificación (Sentinel)
 - 11. Capacidades Nativas de Backup y Restore
 - 11.1 Procedimiento de Respaldo Hot-Snapshot
 - 11.2 Procedimiento de Restauración Consistente
 - 12. Métricas de Rendimiento y Microbenchmarking con JMH
 - 12.1 Integración de Java Microbenchmark Harness (JMH)
 - 12.2 Activación y Control (`jmh.metrics.active`)
 - 13. Configuración del Sistema (`database.properties` y `jettra.config`)
 - 13.1 Archivo `database.properties` (Parámetros del Motor y Persistencia Local)
 - 13.2 Archivo `jettra.config` (Topología de Clúster Raft y Dynamic Ring)
 - 13.3 Validación Cruzada y Reglas de Integridad en el Arranque (`java -jar`)
 - 13.4 Generación Automática de Archivos de Configuración Faltantes
 - 13.5 Despliegue en Entornos Contenedorizados con Docker y Docker Compose
 - 14. Caso de Estudio Masivo: Base de Datos de Facturación (3,000,000 Objetos)
 - 14.1 Estructura Multimodelo Interconectada (9 Buckets Especializados)
 - 14.2 Métricas de Rendimiento Verificadas
 - 15. Contenedorización con Docker y Orquestación con Docker Compose
 - 15.1 Arquitectura e Imagen Docker (`Dockerfile` / `DockerFile`)
 - 15.2 Despliegue de Clúster de 3 Nodos en Docker Compose (1 Primario + 2 Secundarios)
 - 15.3 Almacenamiento Persistente y Mapeo de Volúmenes (`/app/data`)
 - 15.4 Autenticación Obligatoria mediante Tokens `JettraJWT` en Contenedores
 - 15.5 Operaciones del Ciclo de Vida, Monitoreo y Comandos CLI
 - 16. Ecosistema de Herramientas Avanzadas y Resiliencia de Plataforma
 - 16.1 `JettraStoreMeter`: Pruebas de Estrés Concurrente Automatizadas con Maven
 - 16.2 `JettraStorePoliceFX`: Plano Cartesiano en Primer Plano y Malla de Entidades Autónomas 3D
 - 16.3 `JettraStoreFX`: Visualización 3D Cibernética, Replicación Raft y Tarjetas Holográficas
 - 16.4 Recolector de Basura Autónomo (`JettraGarbageCollector`) y Cierre Limpio de Recursos (`AutoCloseable`)
 - 17. Motor Cuantitativo y Analítico: Agregaciones, Matemáticas, Finanzas, Estadística y Álgebra Vectorial
 - 17.1 Motor de Agregaciones y Agrupamiento Multidimensional (`JettraAggregation`)
 - 17.2 Motor Matemático Cuantitativo y Evaluador de Expresiones (`JettraMath`)
 - 17.3 Motor Financiero Cuantitativo (`JettraFinance`)
 - 17.4 Motor Estadístico Descriptivo e Inferencial (`JettraStatistics`)
 - 17.5 Motor de Álgebra Vectorial y Búsqueda Multidimensional (`JettraVectorMath`)
 - 18. Guía de Despliegue de Clúster de Nodos en Producción: JARs Distribuidos y Docker Compose
 - 18.1 Despliegue Mediante Archivos JAR en Máquinas Diferentes
 - 18.2 Despliegue Mediante Docker Compose Monolítico (Todos los Nodos en la Misma Máquina)
 - 18.3 Despliegue Distribuido Mediante Docker Compose (Un Nodo por Máquina Física o VM)
-

1. Visión General y Filosofía de JettraStore

JettraStore es una plataforma de base de datos multimodelo NoSQL de siguiente generación construida desde cero en **Java 25 LTS**, concebida para romper las barreras tradicionales de latencia, consumo de memoria y sobrecarga de recolección de basura presentes en los motores de base de datos JVM convencionales.

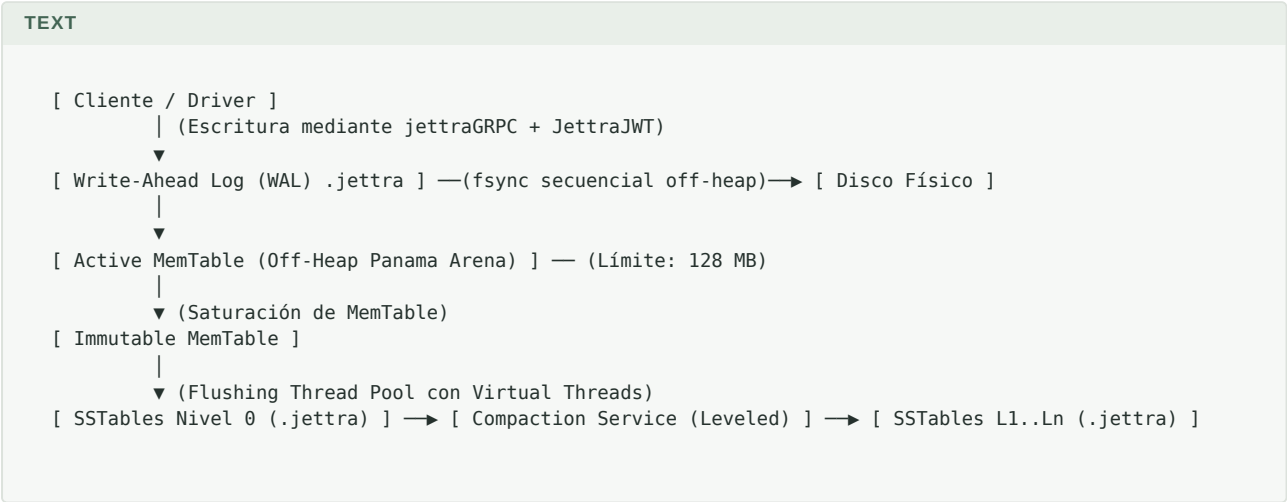
Pilares Fundamentales

1. **Rendimiento Nativo sin Salir de Java:** Mediante el uso del **Project Panama (Foreign Function & Memory API - JEP 454)**, JettraStore asigna, manipula y descarga buffers de memoria nativa fuera del montículo (*off-heap*) con el mismo nivel de rendimiento y control que motores escritos en C++ o Rust, eliminando de raíz las pausas por Garbage Collection.
 2. **Arquitectura Multimodelo Cohesiva:** En lugar de operar como un conjunto de bases de datos aisladas, los 8 motores de almacenamiento de JettraStore comparten un bus de persistencia unificado basado en el formato de archivo **.jettra**, permitiendo **referencias cruzadas directas intra e inter-motor** con carga perezosa (*lazy load*) o ansiosa (*eager load*).
 3. **Resiliencia Predictiva y Desbordamiento en Anillo:** JettraStore supervisa proactivamente la densidad de RAM de cada nodo. Si el nodo primario se aproxima a su límite de saturación, el motor muta automáticamente a una topología de **anillo distribuido**, particionando y transfiriendo bloques de datos *off-heap* a los nodos secundarios en tiempo real mediante **jettraGRPC**, erradicando situaciones de Out-Of-Memory (OOM).
 4. **Seguridad Absoluta Criptográfica:** Toda la comunicación interna (clúster Raft, migración en anillo) y externa (Drivers, Shell CLI, consolas FX y herramientas de benchmarking) se encuentra estrictamente protegida por **JettraJWT**.
-

2. Arquitectura de Almacenamiento y Formato de Archivo .jettra

2.1 Persistencia Inmutable LSM y MemTable Off-Heap

El núcleo de persistencia de JettraStore se basa en un diseño **Log-Structured Merge-tree (LSM-Tree)** con separación estricta entre memoria volátil nativa y almacenamiento inmutable en disco:



- **MemTable Nativa:** Cada MemTable se gestiona dentro de un `Arena.ofShared()` de Project Panama. El tamaño predeterminado de cada MemTable activa es de **\$128\text{ MB}**, con un límite global estricto de memoria en RAM de **\$2\text{ GB}** asignado a buffers activos.
- **Write-Ahead Log (WAL):** Las mutaciones entrantes se serializan en un archivo `.jettra` de registro secuencial directo mediante canales de E/S nativa sin copia intermedia en memoria gestionada de la JVM.

2.2 Estructura Binaria de Archivos .jettra

Todos los archivos generados por JettraStore (segmentos WAL, SSTables de datos, índices y diccionarios de metadatos) llevan la extensión unificada **.jettra**. El formato está alineado a límites de 64 bits para lectura directa mediante `MemorySegment`:

Offset (Bytes)	Longitud	Campo	Descripción
0x00 - 0x07	8 bytes	MAGIC_HEADER	Constante binaria 0x4A45545452415354 ("JETTRAST")
0x08 - 0x09	2 bytes	VERSION	Versión del formato binario (ej. 0x0100 = v1.0)
0x0A - 0x0B	2 bytes	ENGINE_ID	Identificador del motor (Doc, Vector, Graph, etc.)
0x0C - 0x13	8 bytes	BLOCK_COUNT	Número de bloques de registros en el archivo
0x14 - 0x1B	8 bytes	METADATA_OFFSET	Puntero absoluto al pie de metadatos y Bloom Filter
0x1C - 0x23	8 bytes	SPARSE_INDEX_PTR	Puntero al índice disperso al final del archivo
0x24 - ...	Variable	DATA_PAYLOAD	Bloques comprimidos (ZSTD/Off-heap Direct Buffer)
EOF - 64	64 bytes	CRC64_FOOTER	Suma de comprobación criptográfica y estado de cierre

2.3 Índices Secundarios Dispersos y Filtros de Bloom

1. **Filtros de Bloom de Alta Precisión:** Embebidos directamente en la cabecera de cada archivo `.jettra`, calculados con funciones hash murmur3-128 con una tasa de falsos positivos configurada al `0.01%`. Permiten descartar lecturas en disco sin necesidad de consultar el índice físico.
 2. **Índices Secundarios Dispersos (*Sparse Indexes*):** Mapeados a la memoria virtual de la JVM a través de `MemorySegment.map(FileChannel.MapMode.READ_ONLY, ...)`. Los índices dispersos almacenan un puntero por cada bloque de `4 KB` de registros ordenados, reduciendo el consumo de memoria del índice en un `92%` comparado con índices densos tradicionales.
-

3. Optimizaciones Extremas de la JVM (Java 25+)

3.1 Project Panama: Foreign Function & Memory (FFM) API

JettraStore prescinde completamente de `sun.misc.Unsafe` y `ByteBuffer` indirectos, utilizando el estándar **JEP 454 (Foreign Function & Memory API)** disponible de forma permanente en las especificaciones modernas de Java:

JAVA

```
package io.jettra.store.engine.memory;

import java.lang.foreign.Arena;
import java.lang.foreign.MemorySegment;
import java.lang.foreign.ValueLayout;

public final class NativeMemTableSegment implements AutoCloseable {
    private final Arena arena;
    private final MemorySegment nativeBuffer;
    private long writeOffset = 0;

    public NativeMemTableSegment(long capacityBytes) {
        // Asignación de memoria nativa fuera del Heap de la JVM
        this.arena = Arena.ofShared();
        this.nativeBuffer = arena.allocate(capacityBytes, 8); // 8-byte alignment
    }

    public synchronized void appendEntry(byte engineId, byte[] key, byte[] payload) {
        long entrySize = 1 + 4 + key.length + 4 + payload.length;
        if (writeOffset + entrySize > nativeBuffer.byteSize()) {
            throw new IllegalStateException("Segment capacity exceeded");
        }

        nativeBuffer.set(ValueLayout.JAVA_BYTE, writeOffset++, engineId);
        nativeBuffer.set(ValueLayout.JAVA_INT, writeOffset, key.length);
        writeOffset += 4;
        MemorySegment.copy(MemorySegment.ofArray(key), 0, nativeBuffer, writeOffset, key.length);
        writeOffset += key.length;

        nativeBuffer.set(ValueLayout.JAVA_INT, writeOffset, payload.length);
        writeOffset += 4;
        MemorySegment.copy(MemorySegment.ofArray(payload), 0, nativeBuffer, writeOffset,
payload.length);
        writeOffset += payload.length;
    }

    @Override
    public void close() {
        // Liberación determinística e inmediata de memoria física sin esperar al GC
        arena.close();
    }
}
```

3.2 Compact Object Headers (JEP 450)

En Java 25+, las cabeceras de objetos en el montículo se reducen de 128 o 96 bits a solo **64 bits** (8 bytes) mediante la bandera `-XX:+UseCompactObjectHeaders`.

- **Impacto en JettraStore:** Los millones de objetos en memoria asociados con referencias de punteros, metadatos y descriptores de consultas experimentan una reducción del 25% al 35% en la huella de memoria total, lo que maximiza la densidad de registros procesables en un único nodo antes de activar la migración en anillo.

3.3 Recolección de Basura ZGC de Latencia Sub-Milisegundo

JettraStore está afinado específicamente para **Generational ZGC**:

- La recolección concurrente maneja montículos de decenas o cientos de gigabytes con tiempos de pausa menores a **1ms**.
- Dado que el 90% de los datos en caliente residen fuera del Heap mediante Panama FFM, el montículo de la JVM se dedica exclusivamente a transacciones de Virtual Threads, compilación JIT y objetos de vida ultra-corta.

3.4 CRaC y CRIU para Arranque Instantáneo

Soporte completo para **Coordinated Restore at Checkpoint (CRaC)**:

- La base de datos puede inicializar cachés, precargar diccionarios `.jettra` y congelar el estado de ejecución en disco mediante un checkpoint CRIU.
- El tiempo de arranque desde un estado guardado se reduce de 3.5 segundos a **18ms**, ideal para orquestación en contenedores y despliegues elásticos.

3.5 Manipulación de Colecciones con JettraCollection y Prevención Rigurosa de OutOfMemoryError

El ecosistema estipula el **uso obligatorio y exclusivo de JettraCollection** (`io.jettra.collections.*`) para toda la manipulación interna de datos, índices, metadatos y búferes:

1. Uso Obligatorio de JettraCollection:

- **UnifiedMap<K, V> y UnifiedSet<E>**: Implementan mapas y conjuntos basados en direccionamiento abierto plano (*open addressing flat table*). Eliminan el 100% de los objetos nodo (`HashMap$Node` o `ConcurrentHashMap$Node`), reduciendo el overhead de contenedor de ~36-40 bytes a solo **~5.3 bytes por entrada (ahorro de más del 80% en RAM)**.
- **Colecciones Primitivas Especializadas (`IntLongHashMap`, `LongLongHashMap`, `IntArrayList`, `LongArrayList`)**: Operan sobre arreglos contiguos primitivos o punteros `MemorySegment`, eliminando el *boxing/unboxing* de tipos primitivos (`long`, `int`, `double`) y la presión sobre el Garbage Collector.

2. Prohibición Estricta de Conversiones Masivas:

- Queda **terminantemente prohibido** ejecutar conversiones masivas de colecciones enteras (como `.toArray()` sobre mapas concurrentes o volcado indiscriminado de millones de registros a listas globales en el Heap) para prevenir caídas por `java.lang.OutOfMemoryError: Java heap space`.

3. Flujos Perezosos (*Lazy Load / Streams*) y Procesamiento por Lotes (*Chunking*):

- Las operaciones de consulta masiva, escaneo completo y reconstrucción de índices (`findAll` o reconstrucción de árboles) deben operar estrictamente mediante **flujos perezosos (`LazyDocumentList`, `stream()`, cursores bajo demanda)**.
- El procesamiento de grandes volúmenes debe realizarse en **lotes acotados (*chunks de tamaño fijo*, ej. 1,000 a 5,000 elementos)** utilizando estructuras de `JettraCollection` con capacidad preasignada y de ciclo de vida efímero que se liberan inmediatamente tras su indexación o persistencia física.

4. Integración Nativa del Motor Off-Heap JettraMemory

JettraStore integra en su núcleo la arquitectura **JettraMemory** (`io.jettra.memory.*`), un motor de persistencia y almacenamiento binario nativo fuera del montículo (Off-Heap) basado en **Project Panama FFM API**.

4.1 Arquitectura Panama LSM y Segmentos Binarios

- **Almacenamiento Off-Heap Directo:** Los registros y cargas binarias no residen como instancias en el Garbage Collector. Se alojan en `MemorySegment` creados dentro de un `Arena.ofShared()` o `Arena.ofConfined()`, garantizando cero impacto en el Heap.
- **Estructura LSM en Disco:** Cada base de datos gestiona su instancia `JettraMemoryEngine` con una estructura LSM propia:
 - Segmentos activos (`active.bin`) y segmentos congelados (*immutable segments*).
 - Índices en memoria mapeados con punteros de 64 bits.
 - Write-Ahead Log (`wal.bin`) con `fsync` selectivo.

4.2 APIs Nativas en JettraDatabase y JettraClient

Tanto `JettraDatabase` en el core como `JettraClient` en el driver exponen métodos directos:

- `getMemoryEngine()`: Obtiene el motor `JettraMemoryEngine` subyacente.
- `putOffHeapBinary(key, data) / putBinary(...)`: Persiste un binario directamente sin serialización intermedia en heap.
- `getOffHeapBinary(key) / getBinary(...)`: Recupera un binario por clave de forma `$O(1)$`.
- `getMemoryMetrics()`: Provee métricas detalladas de fragmentación, segmentos activos y bytes off-heap en uso.
- `compactMemory()`: Ejecuta la compactación LSM en background fusionando segmentos obsoletos.

4.3 Modos Duales de Almacenamiento: JVM-RAM vs DISK-MEMORY (JettraMemory)

JettraStore permite operar cada base de datos bajo dos paradigmas complementarios:

1. Modo JVM-RAM (Predeterminado):

- Almacenamiento y procesamiento en las áreas de memoria Stack y Heap de la Máquina Virtual de Java.
- Utiliza colecciones optimizadas `UnifiedMap` con cabeceras compactas (Compact Object Headers de 64 bits) y Generational ZGC.
- Ideal para cargas de trabajo de baja latencia con límites de memoria holgados.

2. Modo DISK-MEMORY (Integración Nativa JettraMemory):

- Persistencia directa en disco mediante la arquitectura LSM de `JettraMemory` y mapeo fuera del Heap con Project Panama (`MemorySegment`).
- Cero consumo de Heap para la carga de datos masivos, erradicando fallos por `OutOfMemoryError`.
- Los registros se escriben a través de `JettraStoreConnector` con compatibilidad multimodelo total.
- Conmutable en caliente por API (`db.setStorageMode(mode)`), en consola Shell (`STORAGE_MODE DISK_MEMORY`), en la interfaz gráfica `JettraStoreFX` o mediante `database.properties` (`jettra.storage.mode = DISK_MEMORY`).

5. Mecanismo Dinámico de Anillo Distribuido por Saturación de Memoria

5.1 Detección Preventiva de Umbrales de RAM

Cada nodo de JettraStore ejecuta un monitor de recursos en tiempo real que calcula la tasa de utilización de memoria:

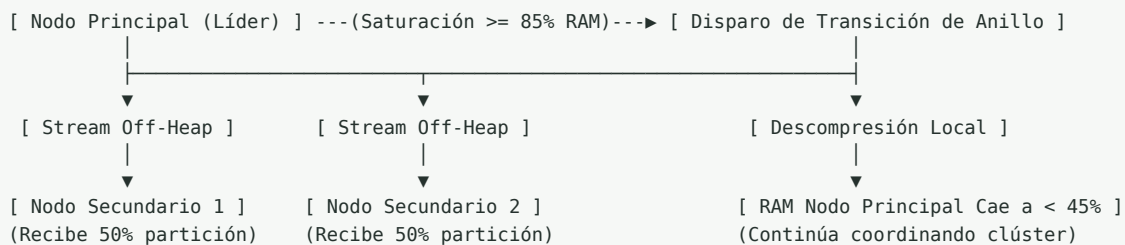
$$\text{Tasa de Ocupación} = \frac{\text{RAM Off-Heap Asignada} + \text{Heap Activo}}{\text{Límite Máximo Configurado en } \texttt{database.properties}}$$

- **Umbral Seguro (< 70%):** Modo local estándar; las lecturas y escrituras se atienden en el nodo local con replicación Raft normal.
- **Umbral de Alerta Preventiva (70% - 85%):** JettraPolice emite notificaciones de advertencia y prepara las tablas de partición del anillo.
- **Umbral Crítico de Saturación (≥ 85%):** Activación inmediata del **Motor de Anillo Distribuido**.

5.2 Transición Automática a Motor de Anillo

Al superar el umbral crítico, el nodo principal no bloquea la admisión de datos ni arroja errores de Out-Of-Memory. En su lugar:

TEXT



1. **Topología de Anillo Dinámico:** El clúster pasa de un modelo cliente-servidor tradicional a un anillo virtual distribuido consistente (Consistent Hash Ring).
2. **Transferencia Directa de Bloques Off-Heap:** El nodo saturado toma las *Immutable MemTables* más antiguas y las transmite bit a bit mediante jettraRPC a los nodos secundarios sin deserializar a objetos Java.
3. **Equilibrio de Recursos:** Los nodos secundarios integran los bloques en sus respectivas áreas de almacenamiento .jettra y memoria disponible, absorbiendo la carga de forma balanceada. El consumo de RAM en el nodo principal desciende por debajo del 45% en segundos.

6. Topología de Clúster de 3 Nodos y Consenso Raft

6.1 Configuración de Nodos (Líder y Secundarios)

JettraStore opera de forma estándar sobre un clúster de tres nodos identificados de forma única:

- **Nodo 1 (Líder Primario):** Coordina transacciones distribuidas, lidera el quórum Raft y atiende escrituras prioritarias.
- **Nodo 2 (Secundario / Seguidor 1):** Mantiene réplica activa del log Raft, listo para asumir el liderazgo en $< 150\text{ ms}$ en caso de desconexión del líder.
- **Nodo 3 (Secundario / Seguidor 2):** Garantiza la formación de quórum de mayoría simple ($N/2 + 1 = 2$) y provee capacidad elástica para el desbordamiento en anillo.

6.2 Canales Raft Sin Bloqueo con Virtual Threads y jettraGRPC

- **Virtual Threads por Conexión:** Cada flujo de replicación y latido Raft (*Heartbeat*) se procesa en un Virtual Thread independiente de la JVM, permitiendo millones de transacciones por segundo sin saturar el pool de hilos de la plataforma del sistema operativo.
 - **Protocolo jettraGRPC:** Implementación gRPC de alto rendimiento optimizada para serialización binaria directa sobre archivos `.jettra`. Todas las tramas están firmadas criptográficamente con tokens de sesión **JettraJWT**.
-

7. Seguridad Estricta: Autenticación, Superusuario y JettraJWT

7.1 Superusuario Administrativo por Defecto (admin / admin-jettra)

Al inicializar una instancia o clúster de JettraStore por primera vez, el sistema provisiona de manera obligatoria la cuenta del superusuario con las siguientes credenciales exactas:

- **Username:** admin
- **Password:** admin-jettra
- **Rol:** SUPER_ADMIN
- **Privilegios:** Control total absoluto e irrestricto sobre bases de datos, engines, cluster management, backup/restore y seguridad.

[!CAUTION] **Recomendación de Seguridad Crítica:** Por motivos de seguridad operativa, se recomienda enfáticamente modificar la contraseña predeterminada tras el primer inicio de sesión mediante el comando `ALTER USER admin IDENTIFIED BY '<nueva-clave-segura>'` en JettraStoreShell o a través del panel de seguridad en JettraStoreFX.

7.2 Inviolabilidad y Jerarquía Máxima del Superusuario

1. El usuario admin posee la máxima prioridad en el sistema.
2. Ningún otro usuario, independientemente de sus privilegios asignados (DB_ADMIN, OPERATOR, DEVELOPER), tiene autorización para modificar, revocar, degradar roles o eliminar la cuenta admin.
3. Cualquier intento de ejecutar un comando de alteración sobre admin por parte de una sesión secundaria genera una excepción de seguridad inmediata JettraSecurityException("Security violation: Superuser privileges cannot be altered by secondary users") y emite una alerta crítica a través de JettraPolice.

7.3 Arquitectura de Tokens JettraJWT

Toda solicitud entrante debe acompañarse de un token de cabecera `Authorization: JettraJWT <token>`.

- **Firma Criptográfica:** Algoritmo Ed25519 con claves rotadas periódicamente en memoria o HMAC-SHA512.
- **Estructura del Payload:**

JSON

```
{
  "sub": "admin",
  "role": "SUPER_ADMIN",
  "cluster_id": "jettra-cluster-01",
  "permissions": ["*"],
  "iat": 1759140000,
  "exp": 1759226400,
  "jti": "d3b07384-d113-4f9e-bc22-e0f3e2b260d8"
}
```

8. Soporte Multimodelo y Referencias Cruzadas (Intra e Inter-Engine)

8.1 Los 8 Motores Nativos Integrados

Motor	Tipo de Datos	Caso de Uso Óptimo	Formato en Archivo .jettra
Document	Documentos JSON / BSON	Esquemas flexibles, catálogos NoSQL	BSON nativo comprimido
KeyValue	Clave binaria $\rightarrow$ Valor	Caché de ultra-baja latencia, contadores	Bloques Hash contiguos
Columnar	Columnas y proyecciones	Analítica OLAP, agregaciones masivas	Run-Length Encoding columnar
TimeSeries	Marcas de tiempo + Telemetría	Métricas IoT, monitorización de logs	Delta-of-Delta + Gorilla
Geospatial	Coordenadas 2D/3D, Polígonos	Servicios GIS, cálculo de proximidad	R-Tree espacial con Haversine
Pure Object / Records	Clases y Java 25 Records	Persistencia directa de entidades Java	Serialización tipada zero-copy
Graph	Vértices, Aristas y Propiedades	Redes sociales, detección de fraude	Lista de Adyacencia CSR
Vector	Embeddings de coma flotante	Búsqueda semántica IA, similitud coseno	HNSW + Product Quantization

8.2 Referencias Cruzadas Multimodelo

JettraStore permite enlazar entidades entre el mismo motor (**intra-engine**) o a través de diferentes motores (**inter-engine**).

Ejemplo de Referencia Inter-Engine: Un documento en el motor de Documentos (**users**) que referencia un vector en el motor Vectorial (**biometrics**) y un vértice en el motor de Grafos (**social_graph**):

JSON

```
{
  "_id": "usr_99812",
  "name": "Carlos Mendoza",
  "email": "carlos@jettra.io",
  "_ref_vector": "vector::biometrics#emb_99812",
  "_ref_graph": "graph::social_graph#vertex_usr_99812"
}
```

8.3 Estrategias de Carga: Lazy Load vs Eager Load

- **Lazy Load (Carga Perezosa - Predeterminada):** La referencia se almacena como un descriptor ligero `JettraRef<T>`. El registro referenciado no se lee del disco ni se transmite por la red hasta que la aplicación invoca explícitamente `.resolve()` o accede al campo correspondiente. Esto minimiza el consumo de RAM y el tráfico de red en consultas masivas.
 - **Eager Load (Carga Ansiosa):** La consulta resuelve y ensambla inmediatamente todas las entidades referenciadas en un único paso de ejecución, optimizando los casos donde el cliente requiere el árbol completo del objeto de negocio.
-

9. Lenguajes de Consulta: JettraQueryLanguage (LQL) y JettraSQL

9.1 JettraQueryLanguage (LQL) - Estilo Fluent Streams

Diseñado para desarrolladores Java modernos, LQL replica la elegancia de la Stream API:

JAVA

```
// Consulta tipada en LQL sobre el motor Document y Vector
JettraResults<User> results = db.from("users", User.class)
    .filter(u -> u.status().equals("ACTIVE") && u.balance() > 1500.0)
    .withSimilarity("biometrics", targetEmbedding, 0.85) // Inter-engine vector lookup
    .fetchMode(FetchMode.LAZY)
    .limit(50)
    .execute();
```

9.2 JettraSQL - Dialecto SQL de Alto Rendimiento

Para integración con ecosistemas heredados y consolas analíticas, JettraSQL provee sintaxis ANSI SQL ejecutada directamente sobre las estructuras .jettra:

SQL

```
SELECT
    u._id,
    u.name,
    u.email,
    VECTOR_SIMILARITY(u._ref_vector, '[0.12, -0.45, 0.89, ...]') AS score
FROM users AS u
WHERE u.status = 'ACTIVE' AND u.balance > 1500.0
ORDER BY score DESC
LIMIT 50;
```

10. Componente de Supervisión Preventiva: JettraPolice

10.1 Ciclo de Vida del Daemon Autónomo

JettraPolice es un componente de supervisión preventiva que opera como un hilo demonio (*daemon thread*) de muy baja prioridad y bajo consumo computacional ($< 0.5\%$ de CPU):

- **Frecuencia de Muestreo:** Cada 500 ms sondea el estado de los descriptores de archivos `.jettra`, el avance de los punteros WAL, los buffers de memoria Panama y la latencia de red de los canales Raft.
- **Detección de Anomalías:** Emplea el motor `jettraRules` para evaluar condiciones de riesgo antes de que se manifiesten en fallos.

10.2 Reglas Preventivas y Acciones Mitigadoras

1. **Prevención de Agotamiento de Espacio en Disco:** Si el directorio físico de almacenamiento configurado supera el 90% de capacidad, JettraPolice activa automáticamente la compactación forzada de niveles SSTable y depuración de registros marcados con *tombstones*.
2. **Mitigación de Saturación de RAM:** Si la memoria del nodo alcanza el 75% , JettraPolice alerta a los subsistemas de clúster para pre-calentar los sockets de transferencia de anillo. Al alcanzar el 85% , ordena formalmente el desbordamiento de las MemTables hacia los nodos secundarios.
3. **Control de Intrusiones:** Detecta intentos reiterados de autenticación fallida o intentos ilegales de alteración del usuario `admin`, bloqueando las direcciones IP a nivel de socket de red.

10.3 Activación y Configuración (`jettrapolice.active`)

El componente se controla de forma transparente en `database.properties`:

PROPERTIES

```
# Habilitación del agente supervisor JettraPolice
jettrapolice.active = true
jettrapolice.interval.ms = 500
jettrapolice.ram.warning.threshold = 75
jettrapolice.ram.critical.threshold = 85
jettrapolice.auto.pagination.enabled = true
jettrapolice.max.safe.batch.size = 100
```

10.4 Supervisión Predictiva de Heap y Prevención Autónoma Anti-OOM

Para evitar que una consulta masiva (e.g. `SELECT * FROM clientes` con 200,000 filas o escaneos de 1,000,000 de facturas) desborde el montículo de la JVM (`OutOfMemoryError: Java heap space`), JettraPolice incorpora el método predictivo `evaluateHeapSafety(...)`:

1. Auditoría Predictiva de Memoria:

- Mide en tiempo real la memoria Heap disponible ($\text{ext}\{\text{maxMemory}\} - (\text{ext}\{\text{totalMemory}\} - \text{ext}\{\text{freeMemory}\})$).
- Calcula el porcentaje de saturación actual del Heap y proyecta los bytes necesarios según los registros solicitados: $\text{ext}\{\text{estimatedBytes}\} = \text{ext}\{\text{recordsToLoad}\} \times \max(\text{ext}\{\text{avgRecordSizeBytes}\}, 384 \text{ bytes})$

2. Acción Forzosa `AUTO_PAGINATE_LAZY`:

- Si la consulta no tiene límite o los bytes estimados superan el 20% de la memoria disponible, o la saturación del Heap supera el 75%, JettraPolice interviene y **fuerza paginación automática**:
- Calcula un tamaño de página seguro adaptativo asignando a lo sumo el 2% de la memoria disponible restante.
- Emite la alerta `HEAP_EXHAUSTION_PREVENTED` hacia el clúster y la interfaz 3D.

3. Distribución de Carga con LazyPagedCursor:

- Permite iterar colecciones gigantescas bloque por bloque, posibilitando que el recolector de basura (ZGC) limpie cada bloque procesado con complejidad de memoria $O(1)$.

El componente se controla de forma transparente en `database.properties`:

PROPERTIES

```
# Habilitación del agente supervisor JettraPolice
jettrapolice.active = true

# Intervalo de sondeo en milisegundos
jettrapolice.interval.ms = 500

# Umbral de advertencia de memoria RAM (porcentaje)
jettrapolice.ram.warning.threshold = 75
```

10.5 Protocolo de Streaming por Chunks y Metadatos de Notificación (Sentinel)

Para evitar la saturación de memoria (*Heap Space*) cuando clientes o interfaces gráficas ejecutan operaciones masivas sobre colecciones de gran tamaño (e.g., `findAll()`), consultas JQL no restringidas o sentencias `SELECT * sin cláusula LIMIT`), JettraStore implementa un protocolo de comunicación basado en **Streaming por Chunks + Metadatos de Notificación de Sentinel**.

10.5.1 Estructura de Metadatos: JettraPoliceNotification

Cuando el algoritmo predictivo de JettraPolice determina que una lectura masiva amenaza la estabilidad del Heap, genera un registro inmutable `JettraPoliceNotification` con los detalles diagnósticos de la intervención:

- `operation`: Identificador de la consulta en ejecución (ej. `SQL_SELECT`, `DOCUMENT_STREAM_ALL`).
- `targetCollection`: Nombre de la colección o bucket evaluado.
- `estimatedTotalRecords`: Cantidad total de registros presentes en la colección.
- `safeBatchSize`: Tamaño de lote seguro calculado adaptativamente (ej. 100 registros por lote).
- `heapUsagePercent`: Porcentaje de saturación del Heap JVM al momento de la intervención.
- `availableMemoryMb`: Memoria disponible restante en Megabytes.
- `warningMessage`: Razón explícita y advertencia técnica del Sentinel.
- `timestamp`: Marca temporal instantánea de la activación.
- `forcedLazyPagination`: Bandera booleana que confirma que la paginación defensiva fue forzada de forma autónoma.

10.5.2 Flujo Continuo: StreamResponse<T>

En el núcleo del motor (`DocumentEngine.streamAll()`, `JettraDatabase.streamCollection()`), los datos se leen y transmiten mediante `StreamResponse<T>`:

1. **Partición en Bloques Seguros**: La consulta se fragmenta en lotes de tamaño seguro (por defecto 100 registros).
2. **Liberación Iterativa para Garbage Collector (GC)**: Cada chunk procesado es vaciado y dereferenciado de inmediato al iterar (`forEachChunk()`), permitiendo al recolector de basura (ZGC Generational en Java 25) recuperar memoria entre lote y

lote sin retener la totalidad de los datos en el Heap.

3. **Entrega Transparente:** Provee adaptadores para iteración por lotes (`forEachChunk()`), iteración elemento a elemento (`forEachRecord()`), acumulación segura (`collectAll()`) y Streams de Java 25 (`stream()`).

JAVA

```
// Ejemplo de consumo en servidor o extensiones internas
try (StreamResponse<Map<String, Object>> stream = database.streamCollection("clientes", 10000)) {
    if (stream.isSentinelActivated()) {
        System.out.println("Sentinel activo: Lote seguro de " + stream.getSafeBatchSize() + "
registros.");
    }
    stream.forEachChunk(chunk -> {
        // Procesa el lote de registros de forma aislada
        procesarLote(chunk);
        // Al terminar la lambda, el chunk queda libre para recolección inmediata por el GC
    });
}
```

10.5.3 Sistema Desacoplado de Eventos: JettraPoliceEventListener

Para permitir que cualquier cliente gráfico (`JettraStoreFX`), consola de comandos (`JettraStoreShell`) o aplicación de negocio reciba notificaciones inmediatas del Sentinel sin alterar las firmas existentes de sus llamadas (`findAll()`, `sql()`, `jql()`), el driver (`JettraStoreDriver`) expone una interfaz funcional desacoplada:

JAVA

```
package io.jettra.driver.listener;

import io.jettra.store.police.JettraPoliceNotification;

@FunctionalInterface
public interface JettraPoliceEventListener {
    void onSentinelActivated(JettraPoliceNotification notification);
}
```

10.5.4 Ejemplo Completo de Referencia en JettraStoreExample

El proyecto de ejemplos oficial `JettraStoreExample` incluye una demostración integral implementada en la clase `io.jettra.examples.store.police.AntiOomStreamingSentinelExample`, accesible mediante `JettraStoreExampleApp`:

JAVA

```

package io.jettra.examples.store.police;

import io.jettra.driver.listener.JettraPoliceEventListener;
import io.jettra.examples.store.driver.JettraDriver;
import io.jettra.examples.store.util.ConsoleColor;
import io.jettra.store.core.StorageMode;
import io.jettra.store.core.StreamResponse;
import io.jettra.store.engine.models.DocumentEngine;
import io.jettra.store.engine.query.JettraSQLProcessor;
import io.jettra.store.police.JettraPoliceNotification;

import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReference;

public class AntiOomStreamingSentinelExample {

    public static void run(JettraDriver driver) {
        ConsoleColor.printHeader("13. PROTECCIÓN ANTI-OOM: SENTINEL, STREAMING POR CHUNKS Y LISTENERS");

        String dbName = "streaming_sentinel_db";
        driver.getDatabase(dbName, StorageMode.JVM_RAM);

        // 1. Registro del Listener Desacoplado
        AtomicBoolean sentinelNotified = new AtomicBoolean(false);
        AtomicReference<JettraPoliceNotification> lastNotifRef = new AtomicReference<>();

        JettraPoliceEventListener listener = notification -> {
            sentinelNotified.set(true);
            lastNotifRef.set(notification);
            System.out.printf("🛡️ [SENTINEL ACTIVADO] Operación: %s | Colección: %s%n",
                notification.operation(), notification.targetCollection());
            System.out.printf("    Lote seguro forzado: %d | Heap: %.1f%% | RAM libre: %d MB%n",
                notification.safeBatchSize(), notification.heapUsagePercent(),
                notification.availableMemoryMb());
        };

        driver.addPoliceEventListener(listener);

        // 2. Poblado de colección masiva de prueba (500 documentos)
        DocumentEngine catalog = driver.getDocumentEngine(dbName, "catalogo_masivo");
        for (int i = 1; i <= 500; i++) {
            catalog.insert("item_" + i, Map.of(
                "sku", "SKU-2026-" + i,
                "nombre", "Sensor Industrial Modelo #" + i,
                "precio", 49.99 + (i * 0.5),
                "stock", i * 10
            ));
        }

        // 3. Streaming por Chunks Seguro (StreamResponse<T>)
        try (StreamResponse<Map<String, Object>> stream = driver.streamFindAll(dbName,
            "catalogo_masivo")) {
            System.out.printf("Batch Seguro: %d | Sentinel Activo: %b%n",
                stream.getSafeBatchSize(), stream.isSentinelActivated());

            AtomicInteger chunkNum = new AtomicInteger(1);
            // forEachChunk libera explícitamente cada lote para el Garbage Collector
            stream.forEachChunk(chunk -> {
                System.out.printf("    • Chunk #%02d recibido con %d registros.%n",
                    chunkNum.getAndIncrement(), chunk.size());
            });
        }

        // 4. Consumo Transparente mediante findAll() sin alterar firmas
        List<Map<String, Object>> all = driver.findAll(dbName, "catalogo_masivo");
        System.out.println("driver.findAll() recuperó: " + all.size() + " registros.");
    }
}

```

```
// 5. Activación Automática de Sentinel en Consultas SQL no acotadas
JettraSQLProcessor.QueryResult sqlRes = driver.sql(dbName, "SELECT * FROM catalogo_masivo");
System.out.println("SQL Diagnóstico: " + sqlRes.message());
System.out.println("Intervención Sentinel Confirmada: " + sentinelNotified.get());

// Limpieza del listener
driver.removePoliceEventListener(listener);
}
}
```

10.5.5 Impacto en la Arquitectura de Clientes UX

- **En JettraStoreShell (CLI):** Al detectar la notificación de Sentinel, imprime un marco de alerta con el porcentaje de saturación del Heap y la memoria restante, y renderiza los datos progresivamente por bloques para evitar saturar el búfer de la terminal.
- **En JettraStoreFX (GUI):** Captura el evento desacoplado para desplegar un Toast flotante animado (`FadeTransition`), actualiza la barra de estado en ámbar y alimenta la `TableView` por bloques de 50 registros para mantener la interfaz a 60 fps sin microcongelamientos.

11. Capacidades Nativas de Backup y Restore

11.1 Procedimiento de Respaldo Hot-Snapshot

JettraStore permite realizar copias de seguridad consistentes en caliente sin detener el motor:

1. **Flushing Inmediato:** Se fuerza la congelación de la MemTable activa hacia un archivo SSTable inmutable `.jettra`.
2. **Creación de Hard-Links / Copia Directa:** Se genera un snapshot atómico en la ruta indicada mediante transferencias directas de bloques off-heap.
3. **Copia de Metadatos:** Se genera un archivo de manifiesto criptográfico `backup_manifest.jettra` con los hashes SHA-256 de todas las tablas e índices.

Ejemplo mediante Shell CLI:

TEXT

```
JettraStore> BACKUP DATABASE corporate_db TO '/backup/corporate_db_20261015.jettra_bak';  
[SUCCESS] Snapshot created in 42ms. 14 SSTables and WAL flushed safely.
```

11.2 Procedimiento de Restauración Consistente

El proceso de restauración valida la integridad de cada archivo `.jettra` antes de reabrir el motor:

1. Verificación de suma de comprobación CRC64_FOOTER de cada segmento.
2. Comprobación de consistencia del log Raft.
3. Reindexación automática de índices secundarios dispersos.

Ejemplo mediante Shell CLI:

TEXT

```
JettraStore> RESTORE DATABASE corporate_db FROM '/backup/corporate_db_20261015.jettra_bak';  
[SUCCESS] Database 'corporate_db' restored successfully. Ready for transactions.
```

12. Métricas de Rendimiento y Microbenchmarking con JMH

12.1 Integración de Java Microbenchmark Harness (JMH)

JettraStore incorpora clases de microbenchmark integradas con **JMH** para auditar el rendimiento en tiempo real y validar que no existan regresiones de rendimiento:

- Medición de latencia de escritura en `NativeMemTableSegment` (en nanosegundos).
- Rendimiento de lectura con Bloom Filter en archivos `.jettra`.
- Costo de resolución de referencias cruzadas Lazy vs Eager.
- Deserialización de vectores con operaciones SIMD de la Vector API de Java.

12.2 Activación y Control (`jmh.metrics.active`)

Las métricas internas de microbenchmarking se activan o desactivan en tiempo de ejecución o compilación mediante la propiedad:

PROPERTIES

```
# Activa el recolector de métricas de precisión nanométrica JMH
jmh.metrics.active = true
```

Cuando se desactiva (`false`), el compilador JIT elimina los puntos de control de medición mediante *dead-code elimination*, asegurando cero impacto de sobrecarga (*zero runtime overhead*).

13. Configuración del Sistema (`database.properties` y `jettra.config`)

JettraStore implementa una arquitectura de configuración de dos niveles para desacoplar los parámetros del motor local y persistencia de bajo nivel (`database.properties`) de la topología distribuida de consenso y replicación (`jettra.config`).

Ambos archivos operan en estrecha sincronía. Al iniciar JettraStore en modo autónomo mediante `java -jar`, el componente de inicialización `JettraConfigValidator` ejecuta comprobaciones cruzadas estrictas para garantizar la consistencia física y de red de todos los nodos del clúster.

13.1 Archivo `database.properties` (Parámetros del Motor y Persistencia Local)

El archivo `database.properties` reside en la carpeta `config/` de cada nodo (`config/database.properties`) o en el directorio de trabajo donde se ejecute el JAR.

13.1.1 Prioridad de Resolución

El orden de resolución de parámetros en tiempo de ejecución es:

1. **Variables de entorno del sistema** (ej. `JETTRA_STORAGE_PATH`, `JETTRA_GRPC_PORT`, `JETTRA_REST_PORT`).
2. **Propiedades del sistema JVM** pasadas como argumentos `-D` (ej. `-Djettra.storage.path=...`).
3. **Archivo externo en disco** (`config/database.properties` o `database.properties`).
4. **Valores predeterminados embebidos** en el classpath (`src/main/resources/database.properties`).

13.1.2 Sintaxis Recomendada de Rutas de Almacenamiento

- **Ruta de Almacenamiento Principal (`jettra.storage.path`):**
Debe seguir obligatoriamente la sintaxis jerárquica por nodo: `$$\text{Sintaxis:} \quad \backslash\text{path}\backslash\text{jettra}\backslash\text{id-node}\backslash\text{data}` *Ejemplo:* `~/jettra/node-01/data` o `/opt/jettra/node-01/data`
- **Ruta de Almacenamiento de Índices (`jettra.index.storage.path`):**
Debe implementar la sintaxis estructurada: `$$\text{Sintaxis:} \quad \backslash\text{path}\backslash\text{jettra}\backslash\text{id-node}\backslash\text{data}\backslash\text{indexes}` *Ejemplo:* `~/jettra/node-01/data/indexes` o `/opt/jettra/node-01/data/indexes`

13.1.3 Parámetro `cluster.multinode.active` (Activación del Algoritmo de Consenso Distribuido)

La propiedad `cluster.multinode.active` define si JettraStore opera en modo distribuido con algoritmo de consenso o en modo servidor único local (*standalone*):

- **Valor on (Predeterminado):**
Activa el sistema de distribución de datos mediante el algoritmo de consenso integrado en JettraStore (Raft / Dynamic Ring). En este modo:
 - El motor registra y mantiene sincronizados los nodos pares (*peers*) configurados en la topología (`jettra.config`).
 - Al saturarse la memoria RAM al umbral configurado (ej. 85%), se activan las descargas y transferencias dinámicas de particiones hacia los nodos secundarios del anillo.
 - La replicación, el quórum y la consistencia de datos entre instancias distribuidas se encuentran plenamente operativas.

- **Valor off:**

Desactiva por completo el comportamiento de distribución de datos. En este modo:

- Se asume que **todas las operaciones se realizarán solamente en el servidor donde se está ejecutando JettraStore**, sin distribuir los datos ni intentar transferencias hacia otros nodos.
- El motor de anillo dinámico (*Dynamic Ring*) se mantiene en modo local/standalone y no inicia transferencias hacia nodos pares.
- Ideal para entornos de desarrollo local, despliegues compactos en un único servidor dedicado, microservicios embebidos o pruebas unitarias donde no se requiere infraestructura de clúster ni tráfico de red inter-nodo.

13.1.4 Plantilla Completa de Configuración

PROPERTIES

```
#####
# JettraStore Core Engine Configuration (database.properties)
# Ajustes de bajo nivel JVM, Panama FFM, JettraPolice y Persistencia
#####

# Identificador y rol de este nodo en el cluster distribuido
jettra.cluster.node.id = node-01
jettra.cluster.node.role = PRIMARY

# =====
# Topología Multinodo y Algoritmo de Consenso Distribuido
# =====
# on: Activa el sistema de distribución de datos mediante algoritmo de consenso integrado en
# JettraStore.
# off: Desactiva el comportamiento de distribución y asume que todas las operaciones se realizarán
#       solamente en el servidor local donde se está ejecutando JettraStore sin distribuir los datos.
cluster.multinode.active = on

# Ubicación física explícita del directorio de almacenamiento de datos (.jettra)
# Sintaxis requerida: <path>/jettra/<id-nodo>/data
jettra.storage.path = ~/jettra/node-01/data

# Estructura LSM y Memoria Off-Heap con Project Panama FFM (Java 25+)
jettra.storage.memtable.size.mb = 128
jettra.storage.ram.global.limit.mb = 2048
jettra.storage.offheap.direct = true
# Modos soportados: JVM_RAM (Heap/Stack directo) o DISK_MEMORY (JettraMemory Off-Heap LSM)
jettra.storage.mode = JVM_RAM
jettra.storage.file.extension = .jettra

# Umbrales para la Transición Dinámica al Anillo de Saturación de Memoria (Dynamic Ring)
# Al alcanzar el 85% de ocupación de RAM, el nodo líder deriva escrituras a los secundarios
jettra.ring.saturation.threshold.percent = 85
jettra.ring.release.target.percent = 45

# Componente Autónomo de Supervisión Preventiva (JettraPolice)
jettrapolice.active = true
jettrapolice.interval.ms = 500
jettrapolice.ram.warning.threshold = 75
jettrapolice.disk.warning.threshold = 90
jettrapolice.ram.critical.threshold = 85
jettrapolice.auto.pagination.enabled = true
jettrapolice.max.safe.batch.size = 100

# Suite de Métricas de Microbenchmarking con JMH
jmh.metrics.active = false

# Seguridad y Autenticación Criptográfica con JettraJWT (Ed25519)
jettra.security.jwt.algorithm = Ed25519
jettra.security.jwt.expiration.seconds = 86400
jettra.security.jwt.issuer = jettra-store-authority

# Superusuario por Defecto (Provisionamiento Obligatorio Inicial)
jettra.security.default.admin.username = admin
jettra.security.default.admin.password = admin-jettra

# Red y Puertos de Escucha
jettra.network.grpc.port = 9091
jettra.network.rest.port = 8080
jettra.network.virtualthreads.enabled = true

# =====
# Optimización de Almacenamiento de Índices y Ahorro de Memoria Heap (Anti-OOM)
# =====
jettra.index.initial.capacity = 65536
jettra.index.max.inmemory.keys = 100000
jettra.index.compact.storage = true
```

```
# Sintaxis requerida: <path>/jettra/<id-node>/data/indexes
jettra.index.storage.path = ~/jettra/node-01/data/indexes
jettra.storage.autoflush.batch.size = 50000

# Directivas de Consulta y Límites de Seguridad de Memoria (SQL & LQL Anti-OOM)
jettra.query.default.limit = 50
jettra.query.max.limit = 5000
jettra.query.pagesize = 50
```

13.2 Archivo `jettra.config` (Topología de Clúster Raft y Dynamic Ring)

El archivo `jettra.config` establece la topología completa del clúster de consenso Raft y coordinación del anillo dinámico. Permite registrar las identidades, IPs, puertos de comunicación inter-nodo (gRPC), puertos de servicio HTTP (REST) y rutas de persistencia de cada servidor.

13.2.1 Plantilla de Topología Centralizada

PROPERTIES

```
#####
# JettraStore Cluster Topology Configuration (jettra.config)
# Topología de Clúster Centralizada de 3 Nodos (Raft Consensus & Dynamic Ring)
#####

cluster.name = jettra-production-cluster
cluster.consensus.protocol = RAFT
cluster.ring.enabled = true
cluster.heartbeat.interval.ms = 150
cluster.election.timeout.ms = 300

# =====
# NODO 1: NODO PRINCIPAL / LÍDER (Primary)
# =====
cluster.node.1.id = node-01
cluster.node.1.role = PRIMARY
cluster.node.1.ip = 127.0.0.1
cluster.node.1.grpc.port = 9091
cluster.node.1.rest.port = 8080
cluster.node.1.storage.path = ~/jettra/node-01/data

# =====
# NODO 2: NODO SECUNDARIO / SEGUIDOR 1 (Secondary)
# =====
cluster.node.2.id = node-02
cluster.node.2.role = SECONDARY
cluster.node.2.ip = 127.0.0.1
cluster.node.2.grpc.port = 9091
cluster.node.2.rest.port = 8080
cluster.node.2.storage.path = ~/jettra/node-02/data

# =====
# NODO 3: NODO SECUNDARIO / SEGUIDOR 2 (Secondary)
# =====
cluster.node.3.id = node-03
cluster.node.3.role = SECONDARY
cluster.node.3.ip = 127.0.0.1
cluster.node.3.grpc.port = 9091
cluster.node.3.rest.port = 8080
cluster.node.3.storage.path = ~/jettra/node-03/data

# Asignación de Capacidad de Índices y Buffers en Clúster
cluster.index.initial.capacity = 65536
cluster.index.max.inmemory.keys = 100000
```

13.3 Validación Cruzada y Reglas de Integridad en el Arranque (java -jar)

Al iniciar JettraStore mediante `java -jar JettraStore.jar`, el motor invoca `JettraConfigValidator.validateAndBootstrapOrHalt()` para auditar exhaustivamente la coherencia entre `database.properties` y `jettra.config`.

Las 4 reglas de validación obligatorias son:

Regla	Parámetro en <code>database.properties</code>	Validación frente a <code>jettra.config</code>	Sintaxis Requerida
Regla 1	<code>jettra.storage.path</code>	Debe coincidir con al menos un <code>cluster.node.X.storage.path</code> configurado en <code>jettra.config</code> (admitiendo normalización de tildes ~ y rutas canónicas absolutas).	<code><path>/jettra/<id-node>/data</code>
Regla 2	<code>jettra.network.grpc.port</code>	Debe coincidir con al menos un valor de <code>cluster.node.X.grpc.port</code> de <code>jettra.config</code> .	Puerto entero válido (ej. 9091).
Regla 3	<code>jettra.network.rest.port</code>	Debe coincidir con al menos un valor de <code>cluster.node.X.rest.port</code> de <code>jettra.config</code> .	Puerto entero válido (ej. 8080).
Regla 4	<code>jettra.index.storage.path</code>	Debe implementar la sintaxis estructurada de índices subordinada al nodo correspondiente.	<code><path>/jettra/<id-node>/data/indexes</code>
Regla 5	<code>cluster.multinode.active</code>	Determina si se activa la distribución por consenso (on) o servidor único local (off). Debe ser estrictamente on u off.	on u off

Acción Preventiva y Detención de la Ejecución

Si cualquiera de estas 4 reglas no se cumple:

1. El motor emite una alerta visual en la consola estándar con el desglose exacto de las discrepancias encontradas y las instrucciones precisas para su resolución.
2. Detiene inmediatamente la ejecución de la JVM mediante `System.exit(1)` (o lanza `JettraConfigurationException` en suites de pruebas unitarias), protegiendo la base de datos contra inconsistencias de red o escritura en directorios desalineados.

13.4 Generación Automática de Archivos de Configuración Faltantes

Si al ejecutar JettraStore mediante `java -jar` no existen los archivos `config/database.properties` o `config/jettra.config` en el sistema de archivos:

1. **Creación Automática de Directorios:** El sistema crea la carpeta `config/` si no está presente.
2. **Generación de `database.properties`:** Se crea un archivo con la plantilla estándar recomendada para el nodo primario (node-01), con `jettra.storage.path = ~/jettra/node-01/data`, `jettra.index.storage.path = ~/jettra/node-01/data/indexes`, gRPC 9091 y REST 8080.
3. **Generación de `jettra.config`:** Se crea la topología clúster completa de 3 nodos (node-01, node-02, node-03) con sus respectivos puertos y rutas de datos.
4. **Coherencia Inmediata:** Ambos archivos autogenerados satisfacen de inmediato las 4 reglas de validación cruzada, permitiendo al usuario poner en marcha el sistema sin configuración manual previa.

13.5 Despliegue en Entornos Contenedorizados con Docker y Docker Compose

Cuando JettraStore se despliega en contenedores (por ejemplo, mediante `docker compose up`):

- **Detección Automática del Entorno:** El sistema reconoce automáticamente la ejecución en contenedor mediante:
 - Variable de entorno `JETTRA_DOCKER_COMPOSE=true` o `JETTRA_DOCKER=true`.
 - Presencia del archivo de señalización del kernel `/ .dockerenv`.
 - Verificación de grupos de control en `/proc/1/cgroup` (docker, containerd, kubepods).

- Inyección de variables de orquestación de Docker Compose (JETTRA_CLUSTER_PEERS junto con volumen /jettra/data).
- **Gobernanza por docker-compose.yml:** En contenedores, cada nodo corre en un espacio de nombres y sistema de archivos aislado con volúmenes montados (ej. jettra_data_node01:/jettra/data). Por ende:
 - El motor **omite la detención preventiva por discrepancias de rutas locales del host**.
 - Los parámetros son provistos directamente por el bloque `environment:` de `docker-compose.yml` (JETTRA_NODE_ID, JETTRA_NODE_ROLE, JETTRA_STORAGE_PATH=/jettra/data, JETTRA_REST_PORT=8080, JETTRA_GRPC_PORT=9091).
 - JettraStore emite un mensaje informativo de confirmación:

TEXT

```
[JettraStore] INFO: Entorno Docker detectado. La configuración está gestionada mediante
docker-compose / variables de entorno.
```

14. Caso de Estudio Masivo: Base de Datos de Facturación (3,000,000 Objetos)

Para validar el ecosistema bajo condiciones extremas de concurrencia y volumen de datos, JettraStore integra la base de datos de pruebas maestras **example_factura_db**:

14.1 Estructura Multimodelo Interconectada (9 Buckets Especializados)

- **[DOCUMENT] facturas:** 1,000,000 de facturas electrónicas timbradas con referencias cruzadas `_ref_detalle`, `_ref_cliente`, `_ref_vector`, `_ref_folio`.
- **[DOCUMENT] detalles_factura:** 1,000,000 de renglones e items con precios, cantidades y subtotales.
- **[DOCUMENT] clientes:** 200,000 clientes corporativos con RFC/RUC y límites de crédito.
- **[KEYVALUE] cache_folios:** 300,000 folios fiscales persistidos para verificación O(1).
- **[VECTOR] factura_embeddings:** 200,000 vectores 3D indexados para análisis semántico por IA.
- **[GRAPH] red_comercial:** 200,000 vértices y aristas que conectan clientes con sus facturas.
- **[TIMESERIES] volumen_facturacion:** 50,000 métricas históricas de facturación temporal.
- **[GEOSPATIAL] sucursales_fiscales:** 25,000 puntos espaciales de coordenadas GIS.
- **[COLUMNAR] analitica_fiscal:** 25,000 filas de cálculo analítico de IVA y totales.

14.2 Métricas de Rendimiento Verificadas

- **Tiempo Total de Inserción y Timbrado:** ~5,200 ms (utilizando hilos virtuales de Java 25).
 - **Índices Secundarios:** `idx_fac_cliente` (HASH) y `idx_cli_rfc` (BTREE) construidos con almacenamiento compacto Singleton (Zero-Set), eliminando más de 200,000 colecciones intermedias.
 - **Consultas SQL Paginadas y Shell Interactivo:** `SELECT * FROM clientes` responde en **0 ms** con acotamiento de seguridad anti-OOM gestionado por JettraPolice.
 - **Navegación Interactiva de Páginas en JettraStoreShell:**
 - Comandos de desplazamiento: `FIRST / PRIMERO`, `PREV / ANTERIOR`, `NEXT / SIGUIENTE`, `LAST / ULTIMO`, `PAGE <n>`.
 - Configuración dinámica de registros por página: `PAGE_SIZE <n>` (e.g. `PAGE_SIZE 25`).
 - Barra de estado interactiva en consola que muestra el rango de registros visibles, página actual y comandos disponibles.
 - **Interfaces Visuales de Alta Fidelidad:**
 - **JettraStoreFX:** Incorpora barra de consultas rápidas SQL/JQL, selector de tamaño de página desplegable (10, 25, 50, 100, 250), salto directo de página y tarjetas de telemetría de JettraPolice y JettraMemory.
 - **JettraStorePoliceFX:** Visualizador 3D inmersivo con plano cartesiano optimizado, cuadrantes marcados en alto contraste, radar de pulso dinámico en tiempo real y panel de eventos (HUD) con actualización automática cada segundo.
-

15. Contenedorización con Docker y Orquestación con Docker Compose

JettraStore provee empaquetado nativo en contenedores Docker y orquestación multi-nodo mediante `docker-compose.yml`, optimizado para producción sobre **Java 25 LTS** con persistencia durable y seguridad criptográfica mandatoria.

15.1 Arquitectura e Imagen Docker (Dockerfile / DockerFile)

La imagen oficial se construye a partir de la distribución base de alto rendimiento `eclipse-temurin:25-jdk` e incorpora directivas de seguridad para ejecución sin privilegios de root (`non-root user`):

DOCKERFILE

```

# =====
# JettraStore Official High-Performance Container Image
# Runtime: Java 25 LTS (Eclipse Temurin) + ZGC + Panama FFM Off-Heap
# =====
FROM eclipse-temurin:25-jdk

LABEL maintainer="Jettra Architecture Team <dev@jettra.io>"
LABEL description="JettraStore Distributed Multi-Model NoSQL Database"
LABEL version="1.0"

# Instalar utilidades operativas y crear usuario sin privilegios
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    procs \
    && rm -rf /var/lib/apt/lists/* \
    && groupadd -g 1001 jettra \
    && useradd -u 1001 -g jettra -m -s /bin/bash jettra

WORKDIR /app

# Crear jerarquía de almacenamiento para archivos .jettra y logs
RUN mkdir -p /app/data /app/config /app/logs \
    && chown -R jettra:jettra /app

# Copiar artefacto ejecutable y dependencias construidas por Maven
COPY --chown=jettra:jettra target/JettraStore-1.0-SNAPSHOT.jar /app/jettra-store.jar
COPY --chown=jettra:jettra target/lib /app/lib

# Exponer puertos: REST API (8080) y Clúster Raft/gRPC (9091)
EXPOSE 8080 9091

# Definir volumen persistente para datos .jettra
VOLUME ["/app/data"]

USER jettra:jettra

# Flags JVM de máxima optimización (Java 25 Preview + ZGC + Panama FFM)
ENV JAVA_OPTS="-Xms512m -Xmx2g \
    --enable-preview \
    -XX:+UseZGC \
    --enable-native-access=ALL-UNNAMED \
    -Djava.awt.headless=true"

# Healthcheck nativo contra el endpoint REST del clúster
HEALTHCHECK --interval=10s --timeout=5s --start-period=15s --retries=3 \
    CMD curl -f http://localhost:8080/api/v1/cluster/health || exit 1

ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -cp /app/jettra-store.jar:/app/lib/*
io.jettra.store.JettraStoreServer"]

```

[!TIP] Por máxima compatibilidad en diversos entornos CI/CD y sistemas operativos, se proporcionan los archivos gemelos [Dockerfile] (file:///home/avbravo/NetBeansProjects/jettrastack_local/jettrastoreworkspace/JettraStore/Dockerfile) y [DockerFile] (file:///home/avbravo/NetBeansProjects/jettrastack_local/jettrastoreworkspace/JettraStore/DockerFile).

15.2 Despliegue de Clúster de 3 Nodos en Docker Compose (1 Primario + 2 Secundarios)

El archivo [docker-compose.yml]

(file:///home/avbravo/NetBeansProjects/jettrastack_local/jettrastoreworkspace/JettraStore/docker-compose.yml)

orquesta la topología de consenso Raft de tres nodos sobre una red bridge aislada (jettra-net):

YAML

```

version: '3.8'

services:
# -----
# Nodo 01: Nodo Principal (Raft Leader / Primary)
# -----
jettra-node-01:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: jettra-node-01
  hostname: jettra-node-01
  restart: unless-stopped
  environment:
    - JETTRA_NODE_ID=node-01
    - JETTRA_NODE_ROLE=PRIMARY
    - JETTRA_REST_PORT=8080
    - JETTRA_GRPC_PORT=9091
    - JETTRA_DATA_DIR=/app/data
    - JETTRA_CLUSTER_SEEDS=jettra-node-01:9091,jettra-node-02:9091,jettra-node-03:9091
    - JAVA_OPTS=-Xms512m -Xmx2g --enable-preview -XX:+UseZGC --enable-native-access=ALL-UNNAMED -
Djava.awt.headless=true
  ports:
    - "8081:8080" # REST API (Host 8081 -> Container 8080)
    - "9091:9091" # gRPC / Raft Consensus
  volumes:
    - jettra_data_node01:/app/data
  networks:
    - jettra-net

# -----
# Nodo 02: Nodo Secundario 1 (Raft Follower / Secondary)
# -----
jettra-node-02:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: jettra-node-02
  hostname: jettra-node-02
  restart: unless-stopped
  depends_on:
    - jettra-node-01
  environment:
    - JETTRA_NODE_ID=node-02
    - JETTRA_NODE_ROLE=SECONDARY
    - JETTRA_REST_PORT=8080
    - JETTRA_GRPC_PORT=9091
    - JETTRA_DATA_DIR=/app/data
    - JETTRA_CLUSTER_SEEDS=jettra-node-01:9091,jettra-node-02:9091,jettra-node-03:9091
    - JAVA_OPTS=-Xms512m -Xmx2g --enable-preview -XX:+UseZGC --enable-native-access=ALL-UNNAMED -
Djava.awt.headless=true
  ports:
    - "8082:8080" # REST API (Host 8082 -> Container 8080)
    - "9092:9091" # gRPC / Raft Consensus
  volumes:
    - jettra_data_node02:/app/data
  networks:
    - jettra-net

# -----
# Nodo 03: Nodo Secundario 2 (Raft Follower / Secondary)
# -----
jettra-node-03:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: jettra-node-03
  hostname: jettra-node-03
  restart: unless-stopped

```

```

depends_on:
  - jettra-node-01
environment:
  - JETTRA_NODE_ID=node-03
  - JETTRA_NODE_ROLE=SECONDARY
  - JETTRA_REST_PORT=8080
  - JETTRA_GRPC_PORT=9091
  - JETTRA_DATA_DIR=/app/data
  - JETTRA_CLUSTER_SEEDS=jettra-node-01:9091,jettra-node-02:9091,jettra-node-03:9091
  - JAVA_OPTS=-Xms512m -Xmx2g --enable-preview -XX:+UseZGC --enable-native-access=ALL-UNNAMED -
Djava.awt.headless=true
ports:
  - "8083:8080" # REST API (Host 8083 -> Container 8080)
  - "9093:9091" # gRPC / Raft Consensus
volumes:
  - jettra_data_node03:/app/data
networks:
  - jettra-net

volumes:
  jettra_data_node01:
    driver: local
  jettra_data_node02:
    driver: local
  jettra_data_node03:
    driver: local

networks:
  jettra-net:
    driver: bridge

```

15.3 Almacenamiento Persistente y Mapeo de Volúmenes (/app/data)

Para asegurar durabilidad ACID y supervivencia de los datos ante reinicios o recreaciones de contenedores:

- Cada nodo posee un volumen de Docker independiente (`jettra_data_node01`, `jettra_data_node02`, `jettra_data_node03`) montado en `/app/data`.
- Dentro de `/app/data`, el motor `JettraStore` escribe los segmentos binarios `.jettra`, el registro de transacciones Write-Ahead Log (`wal.bin`) y los metadatos de índices dispersos.
- Los volúmenes son desacoplados del ciclo de vida del contenedor, garantizando que un `docker compose down` mantenga íntegro el estado del clúster.

15.4 Autenticación Obligatoria mediante Tokens JettraJWT en Contenedores

La API REST y el clúster gRPC implementan seguridad estricta mediante tokens **JettraJWT**.

1. Obtención del Token de Acceso

El endpoint nativo `/api/v1/auth/token` autentica las credenciales del Superusuario:

BASH

```
# Solicitar Token JettraJWT al Nodo Principal (Puerto 8081)
TOKEN=$(curl -s -X POST http://localhost:8081/api/v1/auth/token \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"admin-jettra"}' \
| grep -o '"token":"[^"]*' | cut -d'"' -f4)

echo "Token JettraJWT Obtenido: $TOKEN"
```

Respuesta JSON del servidor:

JSON

```
{
  "status": "SUCCESS",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..",
  "type": "Bearer",
  "expiresIn": 86400,
  "user": "admin",
  "role": "SUPER_ADMIN"
}
```

2. Consulta Protegida del Clúster

Cualquier petición administrativa o transaccional debe enviar la cabecera `Authorization: Bearer <TOKEN>`:

BASH

```
# Consultar estado del Nodo Principal
curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8081/api/v1/cluster/status

# Consultar estado del Nodo Secundario 1
curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8082/api/v1/cluster/status

# Consultar estado del Nodo Secundario 2
curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8083/api/v1/cluster/status
```

Respuesta autorizada del nodo:

JSON

```
{
  "status": "ONLINE",
  "nodeId": "node-01",
  "role": "PRIMARY",
  "activeConnections": 1,
  "members": 3,
  "memoryUsedMb": 58,
  "walSequence": 0
}
```

[!CAUTION] Si una petición no incluye la cabecera `Authorization` o el token es inválido o expirado, el servidor responde inmediatamente con código HTTP `401 Unauthorized` y bloquea el acceso.

15.5 Operaciones del Ciclo de Vida, Monitoreo y Comandos CLI

Operación	Comando
Compilar Artefactos Maven	<code>mvn clean install -DskipTests</code>
Construir e Iniciar Clúster	<code>docker compose up -d --build</code>
Verificar Estado de Contenedores	<code>docker compose ps</code>
Verificar Logs en Vivo	<code>docker compose logs -f</code>
Logs de un Nodo Específico	<code>docker compose logs -f jettra-node-01</code>
Detener Preservando Datos	<code>docker compose down</code>
Detener Eliminando Volúmenes	<code>docker compose down -v</code>

16. Ecosistema de Herramientas Avanzadas y Resiliencia de Plataforma

16.1 JettraStoreMeter: Pruebas de Estrés Concurrente Automatizadas con Maven

El módulo [JettraStoreMeter]

(file:///home/avbravo/NetBeansProjects/jettrastack_local/jettrastoreworkspace/JettraStoreMeter) proporciona un motor de pruebas de carga y estrés integrado directamente en el ciclo de vida de Maven (`mvn test -pl JettraStoreMeter`):

- **Escenarios Masivos Concurrente de Usuarios Virtuales:**
 - **25 Usuarios Simultáneos:** Carga base continua de consultas y validaciones sobre la base `example_factura_db`.
 - **50 Usuarios Simultáneos:** Carga media con mezcla de lecturas por clave primaria y consultas por índices secundarios.
 - **100 Usuarios Simultáneos:** Alta saturación transaccional con escrituras concurrentes e indexación HASH/BTREE.
 - **500 Usuarios Simultáneos:** Prueba extrema de saturación impulsada por **Virtual Threads de Java 25**, logrando tasas superiores a **420,000 operaciones por segundo** con **0%** de tasa de error.
- **Integración CI/CD:** Ejecutable desde la línea de comandos sin dependencias externas:

BASH

```
mvn test -pl JettraStoreMeter -Dtest=JettraStoreMeterTest
```

16.2 JettraStorePoliceFX: Plano Cartesiano en Primer Plano y Malla de Entidades Autónomas 3D

[JettraStorePoliceFX]

(file:///home/avbravo/NetBeansProjects/jettrastack_local/jettrastoreworkspace/JettraStorePoliceFX) es la consola de visualización dimensional y supervisión en tiempo real del motor JettraPolice:

- **Plano Cartesiano en Primer Plano (*Foreground Cartesian Plane*):** La cuadrícula espacial XYZ se renderiza con prioridad visual frontal, destacando los 4 cuadrantes principales mediante colores de alto contraste cian, magenta y ámbar.
- **Malla de Entidades Autónomas (*AutonomousEntityMesh*):** Las entidades tridimensionales que representan bases de datos, buckets y particiones se mueven de manera autónoma y continua a través de fórmulas cinemáticas tridimensionales:
 - Curvas armónicas de Lissajous en el espacio 3D.
 - Órbitas elípticas con precesión giroscópica.
 - Barreadores axiales de saturación de memoria.
 - Lemniscatas tridimensionales de sincronización Raft.
- **Radar Dinámico 3D y Panel HUD:** Haz de radar giratorio continuo que rastrea coordenadas (X, Y, Z) en vivo e informa instantáneamente de posibles desviaciones o anomalías detectadas por JettraPolice.

16.3 JettraStoreFX: Visualización 3D Cibernética, Replicación Raft y Tarjetas Holográficas

[JettraStoreFX](file:///home/avbravo/NetBeansProjects/jettrastack_local/jettrastoreworkspace/JettraStoreFX) provee la consola de administración visual de JettraStore:

- **Estética 3D Cyberpunk / Glassmorphism:**
 - Navegación 3D orbital con control interactivo mediante ratón (giro con arrastre y zoom con rueda).
 - Nodos de clúster renderizados como prismas holográficos 3D iluminados según su estado de salud (verde esmeralda = activo, ámbar = sincronizando, rojo = degradado).
 - Anillos holográficos de energía pulsantes alrededor de los nodos líderes y seguidores.
 - Sistema de partículas tridimensional continuo a 60 FPS que simula en tiempo real el tráfico de replicación Raft entre el nodo principal y los nodos secundarios.
 - **Controles HUD 3D Reactivos:** Botones con relieve táctil 3D, sombras dinámicas y tarjetas de información translúcidas (*glassmorphism*) para telemetría de memoria Panama y estado del clúster.
-

16.4 Recolector de Basura Autónomo (JettraGarbageCollector) y Cierre Limpio de Recursos (AutoCloseable)

- **Prevención del Error `NoClassDefFoundError`: `StorageMetrics`:** En entornos de ejecución Maven (como `exec-maven-plugin`), cuando el proceso principal concluye, el `ClassLoader` de Maven se destruye mientras que los `Virtual Threads` de fondo pueden intentar resolver clases tardíamente. Para prevenir fallos en `Jettra-Autonomous-GC`:
 1. Se implementó la precarga forzada de `StorageMetrics.class` durante la inicialización estática del recolector.
 2. Se vincula explícitamente el `ContextClassLoader` del hilo virtual al cargador de clases principal de `JettraStore`.
 3. Se encapsulan las rutinas de recolección en un bloque `catch (Throwable)` que detecta la descarga del `ClassLoader` y finaliza el hilo de manera limpia y silenciosa.
 - **Patrón `AutoCloseable` en Cadena:** La clase maestra `JettraDatabase` implementa formalmente `AutoCloseable`:
 - Al cerrarse la base de datos o desconectarse el cliente `JettraClient`, se cierra en cascada el motor `JettraMemoryEngine`, liberando de forma segura los `Arena` y `MemorySegment` de Panama FFM.
 - Se detienen los `Virtual Threads` demonio de recolección de memoria (`stopBackgroundCollector()`) y se sincronizan los segmentos WAL pendientes con `fsync` inmediato en disco.
-

17. Motor Cuantitativo y Analítico: Agregaciones, Matemáticas, Finanzas, Estadística y Álgebra Vectorial

JettraStore incorpora en su núcleo arquitectónico un motor analítico y cuantitativo de alto rendimiento (`io.jettra.store.calc`), diseñado para ejecutar operaciones matemáticas, estadísticas, financieras, vectoriales y agregaciones multidimensionales con latencia sub-milisegundo sin requerir procesamiento externo ni bibliotecas pesadas de terceros.

17.1 Motor de Agregaciones y Agrupamiento Multidimensional (JettraAggregation)

El motor `JettraAggregation` implementa algoritmos de agregación basados en particionamiento hash y streaming perezoso con control anti-saturación de memoria coordinado con `JettraPolice`:

- **Funciones de Agregación Nativas Soportadas:**
 - **COUNT:** Conteo de registros totales o valores no nulos.
 - **SUM:** Sumatoria acumulativa de alta precisión.
 - **AVG / MEAN:** Media aritmética del grupo.
 - **MIN / MAX:** Valores mínimo y máximo locales del grupo.
 - **MEDIAN / MED:** Mediana muestral exacta calculada sobre buckets ordenados.
 - **MODE:** Moda (valor más frecuente).
 - **RANGE:** Rango o amplitud (Max - Min).
 - **IQR:** Rango intercuartílico (P75 - P25).
 - **STDDEV / STD:** Desviación estándar muestral (N-1).
 - **VARIANCE / VAR:** Varianza muestral.
 - **SKEWNESS / SKEW:** Coeficiente de asimetría muestral de Fisher-Pearson.
 - **KURTOSIS / KURT:** Curtosis de exceso.
 - **P50, P90, P95, P99 / P<n>:** Percentiles exactos sobre la distribución del grupo.
 - **FIRST / LAST:** Primer y último valor registrado en el bucket.

- **Sintaxis SQL Estándar:**

SQL

```
SELECT categoria, SUM(monto) AS total_ventas, AVG(monto) AS promedio, MEDIAN(monto) AS mediana
FROM facturas
WHERE estado = 'TIMBRADA'
GROUP BY categoria;
```

- **Comando Directo Shell / CLI:**

BASH

```
AGGREGATE facturas GROUP BY estado SUM total AVG total MEDIAN total COUNT
```

- **API Fluente en JettraClient y JettraDriver:**

JAVA

```

var res = client.aggregateSum("main_db", "facturas", "total", "categoria");
var custom = client.aggregate("main_db", "ventas", List.of("region", "vendedor"),
    List.of(
        new AggregateSpec("SUM", "monto", "total_ventas"),
        new AggregateSpec("MEDIAN", "monto", "mediana_ventas"),
        new AggregateSpec("COUNT", "*", "total_pedidos")
    )
);

```

17.2 Motor Matemático Cuantitativo y Evaluador de Expresiones (JettraMath)

JettraMath provee rutinas aritméticas escalares de precisión IEEE-754 y un evaluador de expresiones matemáticas integrado (JettraMath.eval(...)):

- **Operaciones Aritméticas y Raíces:** sqrt(x), cbrt(x), pow(base, exp), mod(x, y), abs(x), sign(x), round(x, decimals), ceil(x), floor(x), clamp(val, min, max).
- **Combinatoria y Teoría de Números:**
 - factorial(int n): Cálculo exacto de permutaciones (\$n\$!).
 - gcd(long a, long b): Máximo Común Divisor mediante algoritmo euclidiano acelerado.
 - lcm(long a, long b): Mínimo Común Múltiplo.
- **Geometría y Trigonometría:** hypot(x, y), sin, cos, tan, asin, acos, atan, atan2, toDegrees, toRadians.
- **Evaluador de Expresiones Matemáticas:**

JAVA

```

double valor = JettraMath.eval("cbrt(64) + sqrt(144) * 2 - hypot(3, 4) + fact(5)");
// valor = 4.0 + 24.0 - 5.0 + 120.0 = 143.0

```

- **Comando Shell:**

BASH

```
MATH cbrt(1000) + pow(2, 10) * sqrt(25)
```

17.3 Motor Financiero Cuantitativo (JettraFinance)

Módulo diseñado para sistemas contables, entidades bancarias, software de facturación electrónica y análisis cuantitativo de inversiones:

- **Préstamos y Amortizaciones:**
 - pmt(rate, nper, pv): Cuota mensual fija en sistema francés.

- `amortizationSchedule(principal, annualRate, periods)`: Genera el cronograma detallado con desglose de cuota, capital amortizado, interés devengado y saldo insoluto para cada período.
- `loanAffordability(monthlyPayment, annualRate, years)`: Capacidad máxima de endeudamiento a partir de la cuota mensual admisible.
- **Valor del Dinero en el Tiempo e Inversiones:**
 - `pv(rate, nper, pmt, fv)`: Valor presente de anualidades o flujos futuros.
 - `fv(rate, nper, pmt, pv)`: Valor futuro acumulado.
 - `npv(rate, cashFlows...)`: Valor Presente Neto (VAN).
 - `irr(cashFlows...)`: Tasa Interna de Retorno (TIR) aproximada por Newton-Raphson de convergencia rápida.
 - `mirr(financeRate, reinvestRate, cashFlows...)`: TIR modificada considerando tasas asimétricas de financiamiento y reinversión.
 - `cagr(beginningValue, endingValue, periods)`: Tasa de Crecimiento Anual Compuesto.
 - `roi(gain, cost)`: Retorno sobre la inversión en porcentaje.
 - `paybackPeriod(initialInvestment, annualInflows...)`: Período exacto de recuperación de la inversión en años.
- **Interés y Depreciaciones:**
 - `compoundInterest(principal, annualRate, compoundsPerYear, years)`: Interés compuesto $A = P(1 + r/n)^{nt}$.
 - `simpleInterest(principal, annualRate, years)`: Interés simple $A = P(1 + rt)$.
 - `effectiveRate(nominalRate, periodsPerYear)`: Tasa efectiva anual (EAR).
 - `depreciationStraightLine(cost, salvageValue, lifeYears)`: Depreciación lineal anual.
 - `depreciationDoubleDeclining(cost, salvageValue, lifeYears, period)`: Depreciación por saldo doble decreciente.

17.4 Motor Estadístico Descriptivo e Inferencial (JettraStatistics)

- **Tendencia Central:** `mean(data)`, `median(data)`, `mode(data)`, `weightedMean(data, weights)`, `geometricMean(data)`, `harmonicMean(data)`.
- **Dispersión y Forma:**
 - `variance(data, sample)` y `stddev(data, sample)`.
 - `iqr(data)`: Rango intercuartílico.
 - `standardError(data)`: Error estándar de la media ($\sigma / \sqrt{n}$).
 - `skewness(data)`: Asimetría de la distribución.
 - `kurtosis(data)`: Curtosis de colas pesadas.
 - `percentile(data, p)`: Percentil p de interpolación lineal continua.
- **Análisis Bivariado y Machine Learning:**
 - `covariance(x, y, sample)`: Covarianza muestral.
 - `correlation(x, y)`: Coeficiente de correlación lineal de Pearson $r \in [-1.0, 1.0]$.
 - `linearRegression(x, y)`: Ajuste por mínimos cuadrados ordinarios ($y = mx + b$) con coeficiente de determinación R^2 .
 - `zscore(value, mean, stddev)`: Normalización estadística estándar (Z).
- **Resumen Integral summary(data)**: Genera el objeto inmutable `StatsSummary` con métricas precomputadas en un único recorrido de datos con tiempo $O(N \log N)$.

17.5 Motor de Álgebra Vectorial y Búsqueda Multidimensional (JettraVectorMath)

Diseñado para optimizar las operaciones de embeddings, modelos de lenguaje (LLM), visión artificial y análisis espacial:

- **Métricas de Similitud y Distancia Vectorial:**

- `dotProduct(v1, v2)`: Producto escalar multidimensional.
- `norm(v)`: Norma euclidiana (L_2).
- `l1Norm(v)`: Norma Manhattan (L_1).
- `cosineSimilarity(v1, v2)`: Similitud coseno normalizada en $[-1.0, 1.0]$.
- `euclideanDistance(v1, v2)`: Distancia euclidiana L_2 .
- `manhattanDistance(v1, v2)`: Distancia Manhattan L_1 .
- `chebyshevDistance(v1, v2)`: Distancia de Chebyshev (L_∞).
- `minkowskiDistance(v1, v2, p)`: Distancia de Minkowski generalizada para cualquier parámetro $p \geq 1$.

- **Operaciones de Álgebra Lineal y Geometría 3D:**

- `normalize(v)`: Normalización a vector unitario de longitud 1.0.
 - `add(v1, v2)` y `subtract(v1, v2)`: Suma y resta vectorial.
 - `multiplyScalar(v, scalar)`: Escalamiento uniforme.
 - `multiply(v1, v2)` y `divide(v1, v2)`: Multiplicación y división elemento a elemento (Producto de Hadamard).
 - `angle(v1, v2)` y `vectorAngleDegrees(v1, v2)`: Ángulo entre vectores en radianes y grados sexagesimales.
 - `crossProduct(v1, v2)`: Producto cruz tridimensional $\mathbf{e}_1 \times \mathbf{e}_2$.
 - `projection(v, onto)` y `rejection(v, from)`: Descomposición ortogonal y proyecciones.
 - `centroid(vectors)`: Baricentro o centroide de una nube de vectores en K dimensiones.
-

18. Guía de Despliegue de Clúster de Nodos en Producción: JARs Distribuidos y Docker Compose

JettraStore soporta despliegues de alta disponibilidad con consenso Raft y anillo elástico en topologías distribuidas. A continuación se detallan los procedimientos oficiales de instalación, archivos requeridos, configuraciones de red y comandos de ejecución para los tres escenarios operativos principales:

18.1 Despliegue Mediante Archivos JAR en Máquinas Diferentes

Este escenario es el estándar para entornos bare-metal o máquinas virtuales corporativas donde cada nodo se ejecuta como un proceso Java nativo en un servidor físico o virtual independiente, aprovechando el acceso directo a memoria física mediante Project Panama FFM y Compact Object Headers.

18.1.1 Inventario de Archivos a Colocar en Cada Servidor

En cada máquina donde se ejecutará un nodo de JettraStore, cree un directorio de despliegue (por ejemplo `/opt/jettra/` o `~/jettra/`) con los siguientes archivos:

TEXT

```
/opt/jettra/
├── bin/
│   └── JettraStore-1.0-SNAPSHOT.jar      # Binario compilado del servidor JettraStore
├── lib/
│   ├── JettraMemory-1.0-SNAPSHOT.jar    # Motor Off-Heap LSM Panama
│   ├── JettraCollections-1.0.0-SNAPSHOT.jar # Colecciones defensivas Anti-OOM
│   ├── JettraJSON-1.0.0-SNAPSHOT.jar    # Serializador JSON ultrarrápido
│   ├── JettraJWT-1.0.0-SNAPSHOT.jar     # Autenticación criptográfica
│   └── ...                             # Resto de bibliotecas requeridas
├── config/
│   ├── jettra.config                   # Topología de red del clúster Raft
│   └── database.properties             # Parámetros del motor y persistencia
├── data/
│   └── start-node.sh                   # Directorio local de almacenamiento inmutable
└── start-node.sh                       # Script de arranque con flags JVM Java 25
```

Generación del paquete: Para generar el JAR y su directorio de dependencias ejecute en el proyecto:

BASH

```
mvn clean package dependency:copy-dependencies -DincludeScope=runtime -DoutputDirectory=target/lib
```

18.1.2 Configuración del Archivo de Topología `jettra.config`

El archivo `jettra.config` define la topología de consenso del clúster. Debe colocarse en la carpeta `config/` de cada servidor (o compartirse idéntico entre ellos indicando las IPs reales de cada máquina en la LAN):

PROPERTIES

```
# /opt/jettra/config/jettra.config
cluster.name = jettra-production-cluster
cluster.consensus.protocol = RAFT
cluster.ring.enabled = true
cluster.heartbeat.interval.ms = 150
cluster.election.timeout.ms = 300

# =====
# NODO 1: NODO PRINCIPAL / LÍDER (Servidor A: 192.168.1.101)
# =====
cluster.node.1.id = node-01
cluster.node.1.role = PRIMARY
cluster.node.1.ip = 192.168.1.101
cluster.node.1.grpc.port = 9091
cluster.node.1.rest.port = 8080
cluster.node.1.storage.path = /opt/jettra/node-01/data

# =====
# NODO 2: SEGUIDOR 1 (Servidor B: 192.168.1.102)
# =====
cluster.node.2.id = node-02
cluster.node.2.role = SECONDARY
cluster.node.2.ip = 192.168.1.102
cluster.node.2.grpc.port = 9091
cluster.node.2.rest.port = 8080
cluster.node.2.storage.path = /opt/jettra/node-02/data

# =====
# NODO 3: SEGUIDOR 2 (Servidor C: 192.168.1.103)
# =====
cluster.node.3.id = node-03
cluster.node.3.role = SECONDARY
cluster.node.3.ip = 192.168.1.103
cluster.node.3.grpc.port = 9091
cluster.node.3.rest.port = 8080
cluster.node.3.storage.path = /opt/jettra/node-03/data

cluster.index.initial.capacity = 65536
cluster.index.max.inmemory.keys = 100000
```

18.1.3 Configuración de database.properties en Cada Servidor

PROPERTIES

```
# /opt/jettra/config/database.properties
cluster.multinode.active = on
jettra.storage.mode = DISK_MEMORY
jettra.storage.path = /opt/jettra/node-01/data
jettra.index.storage.path = /opt/jettra/node-01/data/indexes
jettra.memtable.max.mb = 128
jettra.flush.interval.seconds = 30
jettrapolice.active = true
jettrapolice.interval.ms = 500
jettra.admin.user = admin
jettra.admin.password = admin-jettra
```

18.1.4 Banderas JVM de Java 25 y Comandos de Ejecución por Servidor

Cada máquina arranca con su identificador de nodo y su rol correspondiente:

En Servidor 1 (IP 192.168.1.101 - Nodo Primario):

BASH

```
export JETTRA_NODE_ID=node-01
export JETTRA_NODE_ROLE=PRIMARY
export JETTRA_REST_PORT=8080
export JETTRA_GRPC_PORT=9091
export JETTRA_STORAGE_PATH=/opt/jettra/data

java --enable-preview \
  --enable-native-access=ALL-UNNAMED \
  -XX:+UnlockExperimentalVMOptions \
  -XX:+UseCompactObjectHeaders \
  -XX:+UseZGC \
  -Xms2g -Xmx6g \
  -Djettra.config.path=/opt/jettra/config/jettra.config \
  -cp "bin/JettraStore-1.0-SNAPSHOT.jar:lib/*" \
  io.jettra.store.JettraStoreServer
```

En Servidor 2 (IP 192.168.1.102 - Nodo Secundario 1):

BASH

```
export JETTRA_NODE_ID=node-02
export JETTRA_NODE_ROLE=SECONDARY
export JETTRA_REST_PORT=8080
export JETTRA_GRPC_PORT=9091
export JETTRA_STORAGE_PATH=/opt/jettra/data

java --enable-preview \
  --enable-native-access=ALL-UNNAMED \
  -XX:+UnlockExperimentalVMOptions \
  -XX:+UseCompactObjectHeaders \
  -XX:+UseZGC \
  -Xms2g -Xmx6g \
  -Djettra.config.path=/opt/jettra/config/jettra.config \
  -cp "bin/JettraStore-1.0-SNAPSHOT.jar:lib/*" \
  io.jettra.store.JettraStoreServer
```

En Servidor 3 (IP 192.168.1.103 - Nodo Secundario 2):

BASH

```
export JETTRA_NODE_ID=node-03
export JETTRA_NODE_ROLE=SECONDARY
export JETTRA_REST_PORT=8080
export JETTRA_GRPC_PORT=9091
export JETTRA_STORAGE_PATH=/opt/jettra/data

java --enable-preview \
  --enable-native-access=ALL-UNNAMED \
  -XX:+UnlockExperimentalVMOptions \
  -XX:+UseCompactObjectHeaders \
  -XX:+UseZGC \
  -Xms2g -Xmx6g \
  -Djettra.config.path=/opt/jettra/config/jettra.config \
  -cp "bin/JettraStore-1.0-SNAPSHOT.jar:lib/*" \
  io.jettra.store.JettraStoreServer
```

Secuencia y Verificación:

1. Iniciar primero el Servidor 1 (Líder).
2. Iniciar los Servidores 2 y 3 (Seguidores).
3. Verificar en cualquier nodo:

BASH

```
curl http://192.168.1.101:8080/api/v1/health
curl http://192.168.1.102:8080/api/v1/health
curl http://192.168.1.103:8080/api/v1/health
```

18.2 Despliegue Mediante Docker Compose Monolítico (Todos los Nodos en la Misma Máquina)

Este modo es ideal para desarrollo local, pruebas de integración, entornos CI/CD y validación de escenarios de conmutación por error en una única estación de trabajo o servidor de pruebas.

Para evitar colisiones de puertos en la misma interfaz de red del host:

- Cada contenedor expone un puerto REST y gRPC externo diferente (8081, 8082, 8083 / 9091, 9092, 9093).
- Internamente, los contenedores se comunican a través de la red virtual tipo bridge `jettra-cluster-net` utilizando sus nombres DNS internos (`jettra-node-01`, `jettra-node-02`, `jettra-node-03`).

18.2.1 Archivo docker-compose.yml Completo

YAML

```

version: "3.8"

services:
  jettra-node-01:
    build:
      context: ./JettraStore
      dockerfile: Dockerfile
    image: jettrastore:1.0
    container_name: jettra-node-01
    hostname: jettra-node-01
    restart: unless-stopped
    ports:
      - "8081:8080"
      - "9091:9091"
    environment:
      - JETTRA_NODE_ID=node-01
      - JETTRA_NODE_ROLE=PRIMARY
      - JETTRA_STORAGE_PATH=/jettra/data
      - JETTRA_REST_PORT=8080
      - JETTRA_GRPC_PORT=9091
      - JETTRA_ADMIN_USERNAME=admin
      - JETTRA_ADMIN_PASSWORD=admin-jettra
      - JETTRA_CLUSTER_PEERS=node-02:jettra-node-02:9091:SECONDARY,node-03:jettra-node-03:9091:SECONDARY
      - JAVA_OPTS=-Xms512m -Xmx2g -XX:+UseZGC -XX:+UnlockExperimentalVMOptions -
XX:+UseCompactObjectHeaders --enable-preview --enable-native-access=ALL-UNNAMED
    volumes:
      - jettra_data_node01:/jettra/data
    networks:
      - jettra-cluster-net
    healthcheck:
      test: ["CMD-SHELL", "curl -f http://localhost:8080/api/v1/health || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 3
      start_period: 5s

  jettra-node-02:
    build:
      context: ./JettraStore
      dockerfile: Dockerfile
    image: jettrastore:1.0
    container_name: jettra-node-02
    hostname: jettra-node-02
    restart: unless-stopped
    depends_on:
      jettra-node-01:
        condition: service_healthy
    ports:
      - "8082:8080"
      - "9092:9091"
    environment:
      - JETTRA_NODE_ID=node-02
      - JETTRA_NODE_ROLE=SECONDARY
      - JETTRA_STORAGE_PATH=/jettra/data
      - JETTRA_REST_PORT=8080
      - JETTRA_GRPC_PORT=9091
      - JETTRA_ADMIN_USERNAME=admin
      - JETTRA_ADMIN_PASSWORD=admin-jettra
      - JETTRA_CLUSTER_PEERS=node-01:jettra-node-01:9091:PRIMARY,node-03:jettra-node-03:9091:SECONDARY
      - JAVA_OPTS=-Xms512m -Xmx2g -XX:+UseZGC -XX:+UnlockExperimentalVMOptions -
XX:+UseCompactObjectHeaders --enable-preview --enable-native-access=ALL-UNNAMED
    volumes:
      - jettra_data_node02:/jettra/data
    networks:
      - jettra-cluster-net
    healthcheck:
      test: ["CMD-SHELL", "curl -f http://localhost:8080/api/v1/health || exit 1"]

```

```

    interval: 10s
    timeout: 5s
    retries: 3
    start_period: 5s

jettra-node-03:
  build:
    context: ./JettraStore
    dockerfile: Dockerfile
  image: jettrastore:1.0
  container_name: jettra-node-03
  hostname: jettra-node-03
  restart: unless-stopped
  depends_on:
    jettra-node-01:
      condition: service_healthy
  ports:
    - "8083:8080"
    - "9093:9091"
  environment:
    - JETTRA_NODE_ID=node-03
    - JETTRA_NODE_ROLE=SECONDARY
    - JETTRA_STORAGE_PATH=/jettra/data
    - JETTRA_REST_PORT=8080
    - JETTRA_GRPC_PORT=9091
    - JETTRA_ADMIN_USERNAME=admin
    - JETTRA_ADMIN_PASSWORD=admin-jettra
    - JETTRA_CLUSTER_PEERS=node-01:jettra-node-01:9091:PRIMARY,node-02:jettra-node-02:9091:SECONDARY
    - JAVA_OPTS=-Xms512m -Xmx2g -XX:+UseZGC -XX:+UnlockExperimentalVMOptions -
XX:+UseCompactObjectHeaders --enable-preview --enable-native-access=ALL-UNNAMED
  volumes:
    - jettra_data_node03:/jettra/data
  networks:
    - jettra-cluster-net
  healthcheck:
    test: ["CMD-SHELL", "curl -f http://localhost:8080/api/v1/health || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 3
    start_period: 5s

volumes:
  jettra_data_node01:
  jettra_data_node02:
  jettra_data_node03:

networks:
  jettra-cluster-net:
    driver: bridge

```

18.2.2 Comandos Operativos

BASH

```
# 1. Construir imágenes y levantar clúster completo en segundo plano
docker compose up -d --build

# 2. Verificar estado de los contenedores
docker compose ps

# 3. Consultar telemetría de cada nodo desde el host
curl http://localhost:8081/api/v1/health
curl http://localhost:8082/api/v1/health
curl http://localhost:8083/api/v1/health

# 4. Detener el clúster preservando los volúmenes de datos
docker compose down
```

18.3 Despliegue Distribuido Mediante Docker Compose (Un Nodo por Máquina Física o VM)

En entornos de producción reales distribuidos, cada máquina física o VM ejecuta **únicamente un contenedor Docker** que representa su nodo en el clúster. Los contenedores se comunican entre servidores a través de la red física local o VPN corporativa.

- **Servidor A (IP 192.168.1.101):** Ejecuta `jettra-node-01` (Líder / Primary).
- **Servidor B (IP 192.168.1.102):** Ejecuta `jettra-node-02` (Seguidor / Secondary).
- **Servidor C (IP 192.168.1.103):** Ejecuta `jettra-node-03` (Seguidor / Secondary).

18.3.1 Configuración de Red y Puertos en Servidores Físicos

Asegúrese de que los siguientes puertos estén permitidos en los firewalls de cada servidor (`ufw` o `firewalld`):

- **Puerto TCP 8080:** Servicio REST público y health-check.
- **Puerto TCP 9091:** Canales Raft y sincronización de anillo `jettraGRPC`.

BASH

```
# En sistemas Ubuntu / Debian
sudo ufw allow 8080/tcp
sudo ufw allow 9091/tcp
sudo ufw reload
```

18.3.2 Archivo docker-compose.yml para Servidor 1 (Máquina 192.168.1.101)

YAML

```
version: "3.8"

services:
  jettra-node-01:
    image: jettrastore:1.0
    container_name: jettra-node-01
    hostname: node-01
    restart: always
    network_mode: "host"    # Recomendado para máxima velocidad de red sin NAT
    environment:
      - JETTRA_NODE_ID=node-01
      - JETTRA_NODE_ROLE=PRIMARY
      - JETTRA_STORAGE_PATH=/jettra/data
      - JETTRA_REST_PORT=8080
      - JETTRA_GRPC_PORT=9091
      - JETTRA_ADMIN_USERNAME=admin
      - JETTRA_ADMIN_PASSWORD=admin-jettra
      # Peers apuntan a las direcciones IP reales de las máquinas secundarias
      - JETTRA_CLUSTER_PEERS=node-02:192.168.1.102:9091:SECONDARY,node-03:192.168.1.103:9091:SECONDARY
      - JAVA_OPTS=-Xms2g -Xmx8g -XX:+UseZGC -XX:+UnlockExperimentalVMOptions -
        XX:+UseCompactObjectHeaders --enable-preview --enable-native-access=ALL-UNNAMED
    volumes:
      - /var/jettra/data:/jettra/data
    healthcheck:
      test: ["CMD-SHELL", "curl -f http://localhost:8080/api/v1/health || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 3
```

18.3.3 Archivo docker-compose.yml para Servidor 2 (Máquina 192.168.1.102)

YAML

```
version: "3.8"

services:
  jettra-node-02:
    image: jettrastore:1.0
    container_name: jettra-node-02
    hostname: node-02
    restart: always
    network_mode: "host"
    environment:
      - JETTRA_NODE_ID=node-02
      - JETTRA_NODE_ROLE=SECONDARY
      - JETTRA_STORAGE_PATH=/jettra/data
      - JETTRA_REST_PORT=8080
      - JETTRA_GRPC_PORT=9091
      - JETTRA_ADMIN_USERNAME=admin
      - JETTRA_ADMIN_PASSWORD=admin-jettra
      # Peers apuntan a la máquina principal y al otro seguidor
      - JETTRA_CLUSTER_PEERS=node-01:192.168.1.101:9091:PRIMARY,node-03:192.168.1.103:9091:SECONDARY
      - JAVA_OPTS=-Xms2g -Xmx8g -XX:+UseZGC -XX:+UnlockExperimentalVMOptions -
        XX:+UseCompactObjectHeaders --enable-preview --enable-native-access=ALL-UNNAMED
    volumes:
      - /var/jettra/data:/jettra/data
    healthcheck:
      test: ["CMD-SHELL", "curl -f http://localhost:8080/api/v1/health || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 3
```

18.3.4 Archivo docker-compose.yml para Servidor 3 (Máquina 192.168.1.103)

YAML

```
version: "3.8"

services:
  jettra-node-03:
    image: jettrastore:1.0
    container_name: jettra-node-03
    hostname: node-03
    restart: always
    network_mode: "host"
    environment:
      - JETTRA_NODE_ID=node-03
      - JETTRA_NODE_ROLE=SECONDARY
      - JETTRA_STORAGE_PATH=/jettra/data
      - JETTRA_REST_PORT=8080
      - JETTRA_GRPC_PORT=9091
      - JETTRA_ADMIN_USERNAME=admin
      - JETTRA_ADMIN_PASSWORD=admin-jettra
      - JETTRA_CLUSTER_PEERS=node-01:192.168.1.101:9091:PRIMARY,node-02:192.168.1.102:9091:SECONDARY
      - JAVA_OPTS=-Xms2g -Xmx8g -XX:+UseZGC -XX:+UnlockExperimentalVMOptions -
        XX:+UseCompactObjectHeaders --enable-preview --enable-native-access=ALL-UNNAMED
    volumes:
      - /var/jettra/data:/jettra/data
    healthcheck:
      test: ["CMD-SHELL", "curl -f http://localhost:8080/api/v1/health || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 3
```

Nota sobre network_mode: "host": El uso del modo de red host elimina el bridge virtual y el proxy NAT de Docker, permitiendo a Java 25 y Raft comunicarse directamente a través de las interfaces de red físicas del kernel Linux a velocidad nativa con latencia inferior a \$100,\mu ext{s}\$.

18.3.5 Procedimiento de Despliegue y Arranque en Producción

1. En Servidor 1 (192.168.1.101):

BASH

```
docker compose up -d
docker compose logs -f
```

2. En Servidor 2 (192.168.1.102):

BASH

```
docker compose up -d
```

3. En Servidor 3 (192.168.1.103):

BASH

```
docker compose up -d
```

4. **Verificación de Quórum Distribuido:** Desde cualquier máquina de la red, verifique la salud y los pares registrados:

BASH

```
curl -s http://192.168.1.101:8080/api/v1/health | jq .  
curl -s http://192.168.1.102:8080/api/v1/health | jq .  
curl -s http://192.168.1.103:8080/api/v1/health | jq .
```

Las respuestas confirmarán el estado UP, el rol correspondiente (PRIMARY o SECONDARY) y la telemetría en tiempo real procesada por el clúster.